综合实战项目

报　告　书

班级：2019 级数据智能分析方向

学号：1928424084

姓名：王志龙

日期：2021 年 12 月

目录

目录.....2
1. 确定项目.....6
1.1 项目名称.....6
1.2 项目说明.....6

2. 项目实施.....6
2.1 数据采集.....6
2.11 数据采集原理：.....6
2.12 数据采集操作步骤：.....7
2.13 脚本：.....7
2.14 程序源代码：.....8

2.2 数据清洗.....9
2.21 数据清洗原理:.....9
2.22 数据清洗步骤：.....9
(1)选取所需字段:.....9
（2）搭建数据清洗解析框架：.....9
（3）编写数据清洗代码：.....9
(4)查看数据清洗结果：.....10
2.23 脚本:.....10
2.24 程序源代码:.....11

2.3 数据分析.....12
2.31 数据分析原理：.....12
2.32 数据分析步骤：.....12
（1）数据分析准备：.....12
（2）导入所要分析的总数据并建表：.....12
（3）创建数据分析表进行分析.....14
1、分析用户使用移动端购买还是 PC 端购买，及移动端和 PC 端的用户比例.....14
2、分析用户评论周期（收到货后，一般多久进行评论）.....15
3、分析会员级别（判断购买此商品的用户级别）.....15
4、分析评论中出现的词汇.....16
5、分析评论创建与时间段的关系.....17
6、分析评论量.....18
7、分析评论创建与月份的关系.....18
8、评论创建与年份的关系.....19
9、商品评分.....19

2.4 数据导出.....20
2.41 数据导出原理：.....20
2.42 数据导出步骤：.....20

（1）数据导出准备：..... 20
（2）在 MySQL 中建表：..... 20
1、commentsql 表..... 21
2、creationhour 表..... 21
3、creationmonth 表..... 21
4、creationyeasqlr 表..... 21
5、dayssql 表..... 21
6、ismobile 表..... 21
7、kwcloud 表..... 21
8、scoresql 表..... 21
9、userlevelsql 表..... 21
（3）数据导出（hive->MySQL）：..... 21

（4）在 MySQL 中进行表的查看验证是否成功导入：..... 22
1、对 MySQL 中 commentsql 分析表的查看:..... 22
2、对 MySQL 中 creationhour 分析表的查看:..... 22
3、对 MySQL 中 creationmonth 分析表的查看:..... 23
4、对 MySQL 中 creationyearsql 分析表的查看:..... 23
5、对 MySQL 中 dayssql 分析表的查看:..... 24
6、对 MySQL 中 scoresql 分析表的查看:..... 24
7、对 MySQL 中 ismobile 分析表的查看:..... 24
8、对 MySQL 中 userlevelsql 分析表的查看:..... 24
9、对 MySQL 中 kwcloud 分析表的查看:..... 25

2.5 数据可视化..... 26
2.51 数据可视化的原理..... 26
2.52 数据可视化的步骤..... 26
（1）在 model 层创建 Java 类：..... 26
（2）在 Dao 层创建接口：..... 26
（3）创建 mapper 层：..... 27
（4）在 Service 层创建接口:..... 27
（5）实现 Service 层接口：..... 27
（6）在 Controller 层创建 Java 类：..... 28
（7）在 templates 创建 html 来显示图表：..... 29

2.5.3 数据可视化的界面分析..... 31
（1）Comment 页面：..... 31
（2）CreationHour 页面：..... 31
（3）CreationMonth 页面：..... 32
（4）CreationTime 页面：..... 32
（5）Days 页面：..... 33
（6）Ismobile 页面：..... 33
（7）KwCloud 页面：..... 34
（8）Score 页面：..... 34

（9）Userlevel 页面：..... 35
3. 项目总结.....36

4. 附录（源代码）.....37
CreationHour 代码：.....37
（1）CreationHour.java..... 37
（2）CreationHourDao.java..... 37
（3）CreationHour.Mapper.....37
（4）CreationHourService.java..... 38
（5）CreationHourServiceImpl.java.....38
（6）CreationHourController.java.....39
（7）view-CreationHour.html..... 40
CreationMonth 代码：.....42
（1）CreationMonth.java.....42
（2）CreationMonthDao.java..... 42
（3）CreationMonthMapper..... 42
（4）CreationMonthService.java..... 43
（5）CreationMonthServiceImpl.java.....43
（6）CreationMonthController.java.....44
（7）view-CreationMonth.html.....45
CreationTime 代码：.....47
（1）CreationTime.java.....47
（2）CreationTimeDao.java..... 47
（3）CreationTimeMapper..... 47
（4）CreationTimeService.java..... 48
（5) CreationTimeServiceImpl.java.....48
（6）CreationTimeController.java.....49
（7）view-CreationTime.html.....50

Days 代码：.....52
（1）Days.java..... 52
（2）DaysDao.java.....52
（3）DaysMapper.....52
（4）DaysService.java.....53
（5）DaysServiceImpl.java..... 53
（6）DaysController.java..... 54
（7）view-Days.html..... 55

IsMobile 代码：.....57
（1）IsMobile.java.....57
（2）IsMobileDao.java.....57
（3）IsMobileMapper.....57
（4）IsMobileService.java.....58
（5）IsMobileServiceImpl.java.....58
（6）IsMobileController.java.....59
（7）view-IsMobile.html.....60

KwCloud 代码：.....62
（1）KwCloud.java.....62
（2）KwCloudDao.java.....62
（3）KwCloudMapper.....62
（4）KwCloudService.java.....63
（5）KwCloudServiceImpl.java.....63
（6）KwCloudController.java.....64
（7）view-KwCloud.html.....65

Score 代码：.....67
（1）Score.java.....67
（2）ScoreDao.java.....67
（3）ScoreMapper.....67
（4）ScoreService.java.....68
（5) ScoreServiceImpl.java.....68
（6）ScoreControllerjava.....69
（7）view-Score.html.....70

UserLevel 代码：.....72
（1）Userlevel.java.....72
（2）UserlevelDao.java.....72
（3）UserlevelMapper.....72
（4）UserlevelService.java.....73
（5）UserlevelServiceImpl.java.....73
（6）UserlevelController.java.....74
（7）view-UserLevel.html.....75
HelloContro 代码：.....77
JsonResult 代码：.....78
分词代码：.....79

1. 确定项目

1.1 项目名称

基于京东商城商品评论信息数据分析

1.2 项目说明

通过采集电子商务网站数据、利用数据清洗、数据分析、数据可视化技术完成整体项目的开发工作。任务包含环境准备：搭建 Hadoop 平台及大数据分析工具安装、采集数据：采集电商网站交易及评论数据等、清洗数据：开发 MapReduce 程序清洗评论数据、离线分析：利用 Hive 数据分析工具完成数据统计、数据导出：利用 Sqoop 将 HDFS 数据导出关系数据库、数据可视化：开发 springboot+ECharts 完成数据图表展示结果。

2. 项目实施

2.1 数据采集

2.1.1 原理：

网络爬虫，也叫网络蜘蛛（spider），是一种用来自动浏览万维网的网络机器人。其目的一般为编纂网络索引。网络搜索引擎等站点通过爬虫软件更新自身的网站内容或其对其他网站的索引。网络爬虫可以将自己所访问的页面保存下来，以便搜索引擎事后生成索引供用户搜索。爬虫访问网站的过程会消耗目标系统资源。不少网络系统并不默许爬虫工作。因此在访问大量页面时，爬虫需要考虑到规划、负载，还需要讲“礼貌”。 不愿意被爬虫访问、被爬虫主人知晓的公开站点可以使用 robots.txt 文件之类的方法避免访问。这个文件可以要机器人只对网站的一部分进行索引，或完全不作处理。互联网上的页面极多，即使是最大的爬虫系统也无法做出完整的索引。因此在公元 2000 年之前的万维网出现初期，搜索引擎经常找不到多少相关结果。现在的搜索引擎在这方面已经进步很多，能够即刻给出高质量结果。爬虫还可以验证超链接和 HTML 代码，用于网络抓取。

2.1.2 操作步骤：

（1）分析所爬取网站的源码结构：

截图

（2）搭建爬虫代码框架：

（3）编写数据采集代码：

代码段

（4）查看数据情况：

1.脚本：

cd /data/

mkdir /data/edu1

2.切换目录到/data/edu1 目录下，使用 wget 命令，下载 webmagic 爬虫项目所依赖的 lib 包

cd /data/edu1

wget http://192.168.1.100:60000/allfiles/edu1/webmagic-0.7-libs.tar.gz

将 webmagic-0.7-libs.tar.gz 压缩包，解压缩。

tar -xzvf webmagic-0.7-libs.tar.gz

程序源代码：

```
package com.htu.WebMagic;

import us.codecraft.webmagic.Page;
import us.codecraft.webmagic.Request;
import us.codecraft.webmagic.Site;
import us.codecraft.webmagic.Spider;
import us.codecraft.webmagic.pipeline.ConsolePipeline;
import us.codecraft.webmagic.pipeline.JsonFilePipeline;
```

```
import us.codecraft.webmagic.processor.PageProcessor;
import us.codecraft.webmagic.scheduler.QueueScheduler;
 /**
  *
  * @在 WebMagic 里，实现一个基本的爬虫只需要编写一个类，
  * 实现 us.codecraft.webmagic.processor.PageProcessor 接口即可。
  * 这个类基本上包含了抓取一个网站，需要写的所有代码。实现接口后，
  * 重写里面的 getSite 和 process 方法
  *
  */
public class GetComments implements PageProcessor {
    private Site site = Site.me()
                .setDomain("{PaChongSite}")
                .setSleepTime(2000)

        .setUserAgent("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_2) AppleWebKit/537.31 (KHTML, like Gecko) Chrome/26.0.1410.65 Safari/537.31");
    @Override
    public Site getSite() {
        return site;
    }

    @Override
    public void process(Page page) {
        page.putField("ten",page.getJson().toString());
```

```
    }

    public static void main(String args[])
    {
       String url_init    =
"https://club.jd.com/comment/productPageComments.action?callback=fetc
hJSON_comment98&productId=1003979&score=0&sortType=5&page=0&pageSize
=10&isShadowSku=0&fold=1";
       String url_pattern1 =
"https://club.jd.com/comment/productPageComments.action?callback=fetc
hJSON_comment98&productId=1003979&score=0&sortType=5&page=";
       String url_pattern2="&pageSize=10&isShadowSku=0&fold=1";
       String output      = "/home/hadoop/data/";

       QueueScheduler scheduler = new QueueScheduler();

       Spider spider = Spider.create(new GetComments())
               .addUrl(url_init)
               .setScheduler(scheduler)
               .addPipeline(new JsonFilePipeline(output))
               .addPipeline(new ConsolePipeline());

       for (int i = 0; i < 100; i++) {
           Request request = new Request();
           request.setUrl(url_pattern1 + i+url_pattern2);
           scheduler.push(request, spider);
       }
       spider.thread(5).run();
```

```
    }
}
```

2.2 数据清洗

2.2.1 原理：

数据清洗是对数据进行重新审查和校验的过程，目的在于删除重复信息、纠正存在的错误，并提供数据一致性。数据清洗从名字上也看的出就是把“脏”的“洗掉”，指发现并纠正数据文件中可识别的错误的最后一道程序，包括检查数据一致性，处理无效值和缺失值等。因为数据仓库中的数据是面向某一主题的数据的集合，这些数据从多个业务系统中抽取而来而且包含历史数据，这样就避免不了有的数据是错误数据、有的数据相互之间有冲突，这些错误的或有冲突的数据显然是我们不想要的，称为“脏数据”。我们要按照一定的规则把“脏数据”“洗掉”，这就是数据清洗。而数据清洗的任务是过滤那些不符合要求的数据，将过滤的结果交给业务主管部门，确认是否过滤掉还是由业务单位修正之后再进行抽取。不符合要求的数据主要是有不完整的数据、错误的数据、重复的数据三大类。数据清洗是与问卷审核不同，录入后的数据清洗一般是由计算机而不是人工完成。

2.2.2 操作步骤：

2.2.2.1 选择所需字段

例如，comments 中详细内容：
view plain copy
id": 10432588299,
guid": "6c1d83b1-ac45-4189-a041-774eaff87df9",
content": "割手，相当的割手，无语了", //评论内容 √
creationTime": "2017-05-22 23:37:24", //写评论的时间 √
isTop": false, //是否置顶
referenceTime": "2017-05-20 18:35:11",//收货时间 √
firstCategory": 9987, //第一分类 √
secondCategory": 653, //第二分类 √
thirdCategory": 655, //第三分类 √
replyCount": 0,
score": 3, //打分 √

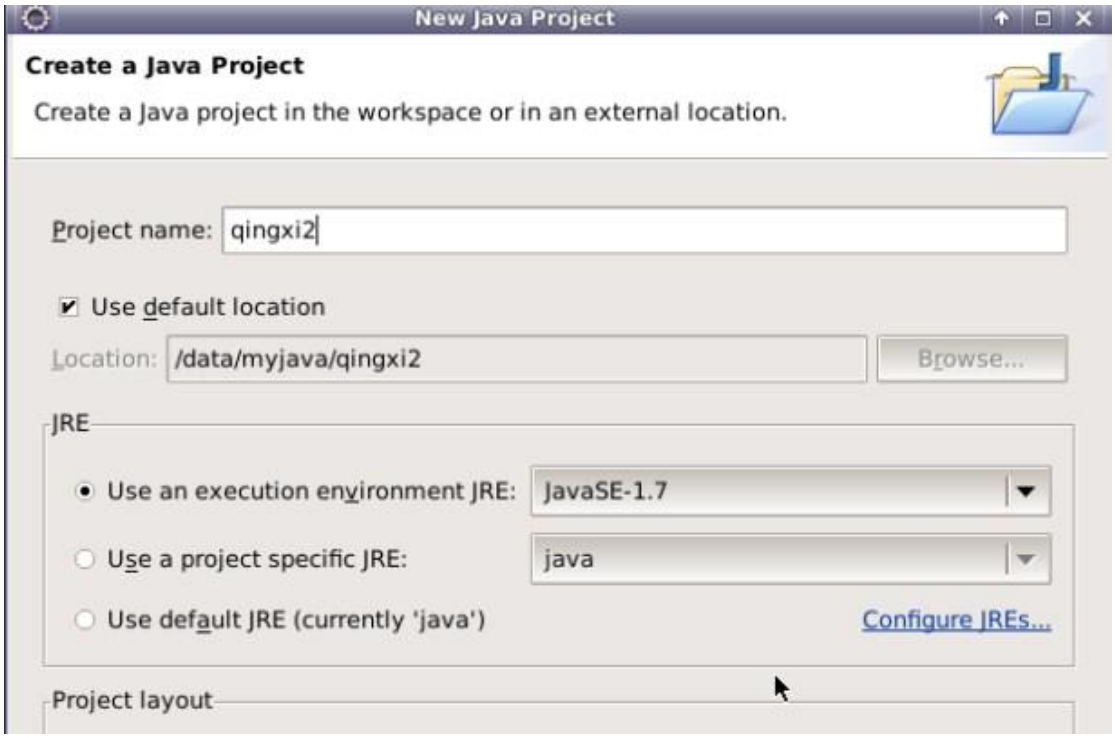
nickname": "j***柜",　　//昵称 √
userClient": 2,
productColor": "碳黑色",
productSize": "32GB",
userLevelName": "金牌会员",　　//会员级别√
plusAvailable": 0,
productSales": [
  {
    "dim": 3,
    "saleName": "选择套装",
    "saleValue": "官方标配"
  }
,
userClientShow": "来自京东 iPhone 客户端",//评论设备
isMobile": true,　　//是否移动端
days": 2,　　//评论时间距【收货/下单】时间多长时间
afterDays": 0
在这段 Json 中，对我们有用的是，如下一些字段
view plain copy
"id": 10432588299,
"guid": "6c1d83b1-ac45-4189-a041-774eaff87df9",
"content": "割手，相当的割手，无语了", //评论内容√
"creationTime": "2017-05-22 23:37:24", //写评论的时间√
"isTop": false,　　//是否置顶
"referenceTime": "2017-05-20 18:35:11", //收货时间√
"score": 3,　　//打分√
"nickname": "j***柜",　　//昵称 √
"userLevelName": "金牌会员",　　//会员级别√
"userClientShow": "来自京东 iPhone 客户端",//评论设备
"isMobile": true,　　//是否移动端
"days": 2,　　//评论时间距【收货/下单】时间多长时间
当然，判断字段是否对我们有用，判断依据是根据需求来定的。后续做的一些需求，会用到哪些字段，此处就会采集哪些字段。

2.2.2.2 搭建数据清洗解析框架：

1.切换目录到/data/目录下，创建名为 edu2 的目录
view plain copy
```
cd /data/
```
view plain copy
```
mkdir /data/edu2
```
2.切换目录到/data/edu2 目录下，使用 wget 命令，下载项目所依赖的 lib 包

view plain copy
cd /data/edu2
view plain copy
wget http://192.168.1.100:60000/allfiles/edu2/hadoop2lib.tar.gz
view plain copy
wget http://192.168.1.100:60000/allfiles/edu2/fastjson-1.2.31.jar
将 hadoop2lib.tar.gz 压缩包，解压缩。
view plain copy
tar -xzvf hadoop2lib.tar.gz
3.打开 eclipse，新建 Java Project

将项目命名为 qingxi2



4.右键项目名，新建一个目录，命名为 libs 用于存储项目依赖的 jar 包

将/data/edu2/hadoop2lib 目录下，所有的 jar 包，拷贝到项目下的 libs 目录下。

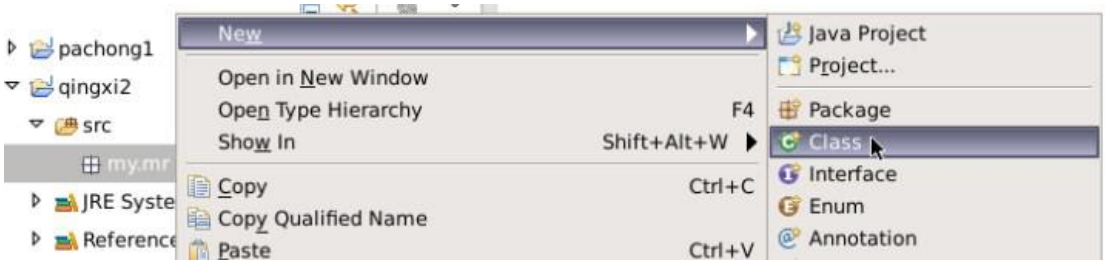
将/data/edu2/目录下，fastjson-1.2.31.jar，拷贝到项目下的 libs 目录下。

选中 libs 下，所有的 jar 文件，依次点击“Build Path” => "Add to Build Path"

5.右键 src，点击 "New" => "Package"，新建一个包

将包命名为 my.mr

右键包名，依次点击“New” => “Class”



并将类命名为 QingXiJson

这样清洗过程的框架搭建完毕，下面开始编写代码实现功能。

2.2.2.3 编写数据清洗代码：

代码所实现的需求，是使用 MapReduce 解析 Json 文件，最终输出格式化的文本文件。

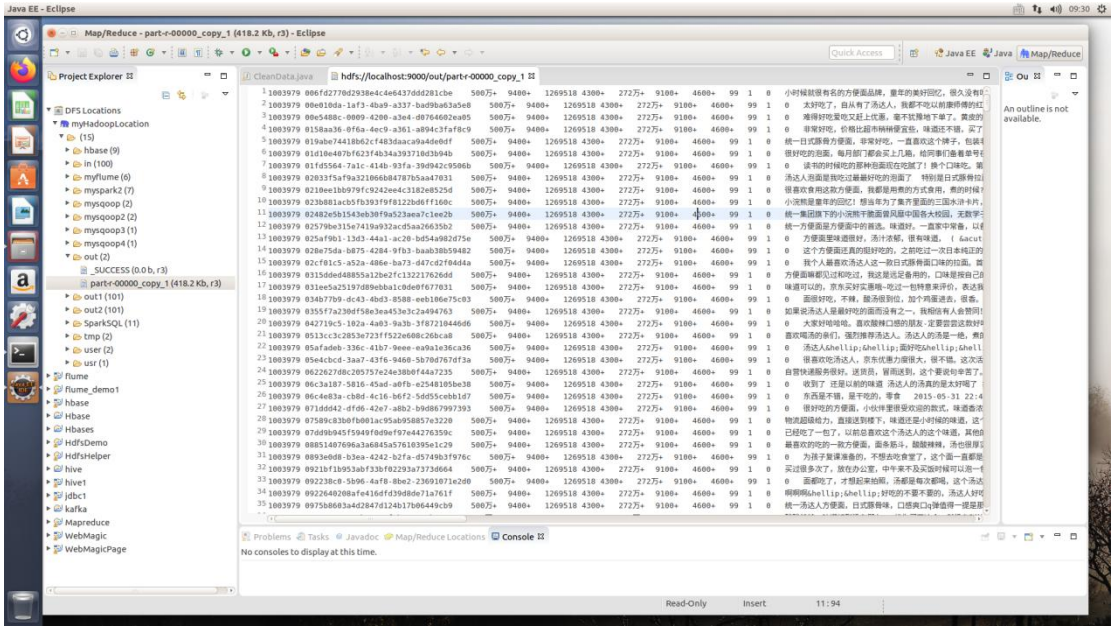
通过分析可以知道，此处只用 Map 任务即可实现具体功能，所以可以省去 Reduce 任务。

完整代码如下：

```
import java.io.IOException;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import com.alibaba.fastjson.JSONArray;
import com.alibaba.fastjson.JSONObject;
public static class doMapper extends Mapper<Object, Text, Text, Text> {
   @Override
   protected void map(Object key, Text value, Context context)
         throws IOException, InterruptedException {
      String initJsonString = value.toString();
      JSONObject initJson = JSONObject.parseObject(initJsonString );
       if (!initJsonString.contains("productCommentSummary") && !initJsonString.contains("comm
ents")) {
         return;
      }

      JSONObject myjson = initJson.getJSONObject("ten");

       JSONObject productCommentSummary = myjson.getJSONObject("productCommentSummar
y");
      String productId      = productCommentSummary.get("productId").toString();
      String commentCount    = productCommentSummary.get("commentCount").toString();
      String goodCount       = productCommentSummary.get("goodCount").toString();
      String generalCount    = productCommentSummary.get("generalCount").toString();
      String poorCount       = productCommentSummary.get("poorCount").toString();
      String goodRateShow    = productCommentSummary.get("goodRateShow").toString();
      String generalRateShow = productCommentSummary.get("generalRateShow").toString();
      String poorRateShow    = productCommentSummary.get("poorRateShow").toString();
      /* comments 包括十条评论 */
      JSONArray comments = myjson.getJSONArray("comments");
      for (int i = 0; i < comments.size(); i++) {
         JSONObject comment    = comments.getJSONObject(i);
         String guid           = comment.getString("guid");
         String content        = comment.getString("content").replace('\n', ' ');
         String creationTime   = comment.getString("creationTime");
         String score          = comment.getString("score");
         String nickname       = comment.getString("nickname");
         String userLevelName  = comment.getString("userLevelName");
         String userClientShow = comment.getString("userClientShow");
         String isMobile       = comment.getString("isMobile");
```

```
        String days          = comment.getString("days");
        StringBuilder sb = new StringBuilder();

        sb.append(productId);        sb.append("\t");
        sb.append(commentCount);     sb.append("\t");
        sb.append(goodCount);        sb.append("\t");
        sb.append(generalCount);     sb.append("\t");
        sb.append( poorCount );      sb.append("\t");
        sb.append( goodRateShow );   sb.append("\t");
        sb.append( generalRateShow ); sb.append("\t");
        sb.append( poorRateShow );   sb.append("\t");
        sb.append( guid );           sb.append("\t");
        sb.append( content );        sb.append("\t");
        sb.append( creationTime );   sb.append("\t");
        sb.append( score );          sb.append("\t");
        sb.append( nickname );       sb.append("\t");
        sb.append( userLevelName );  sb.append("\t");
        sb.append( userClientShow ); sb.append("\t");
        sb.append( isMobile );       sb.append("\t");
        sb.append( days );
        String result = sb.toString();
        context.write(new Text(result), new Text(""));
    }
  }
}
```

(4)查看数据清洗结果：



2.3 数据分析

2.3.1 原理：

Hive 是基于 Hadoop 的一个数据仓库工具，可以将结构化的数据文件映射为一张数据库表，并提供简单的 SQL 查询功能，可以将 SQL 语句转换为 MapReduce 任务进行运行。其优点是学习成本低，可以通过类 SQL 语句快速实现简单的 MapReduce 统计，不必开发专门的 MapReduce 应用，十分适合数据仓库的统计分析。Hive 构建在基于静态批处理的 Hadoop 之上，由于 Hadoop 通常都有较高的延迟并且在作业提交和调度的时候需要大量的开销。因此，Hive 并不适合那些需要低延迟的应用，它最适合应用在基于大量不可变数据的批处理作业，例如，网络日志分析。Hive 的特点是：可伸缩（在 Hadoop 集群上动态的添加设备）、可扩展、容错、输出格式的松散耦合。Hive 将元数据存储在关系型数据库（RDBMS）中，比如 MySQL、Derby 中。

2.3.2 操作步骤：

2.3.2.1 数据分析准备：

脚本：create table pinglun (
productid string,
commentcount int,
goodcount int,
generalcount int,
poorcount int,
goodrateshow float,
generalrateshow float,
poorrateshow float,
guid string,
content string,
creationtime string,
score int,
nickname string,
userclient string,
mobileVersion string,
days int
) row format delimited
fields terminated by '\t';

```
hive> create table pinglun (productid string,commentcount int,goodcount int,generalcount int,
    > poorcount int,goodrateshow float,generalrateshow float,poorrateshow float,guid string,content string,
    > creationtime string,score int,nickname string,userlevelname string,userclientshow string,ismobile string,
    > days int) row format delimited fields terminated by '\t';
OK
Time taken: 0.417 seconds
hive>
```

创建成功后，查看 pinglun 表的表结构：

view plain copy

desc pinglun;

```
hive> desc pinglun;
OK
productid               string
commentcount            int
goodcount               int
generalcount            int
poorcount               int
goodrateshow            float
generalrateshow         float
poorrateshow            float
guid                    string
content                 string
creationtime            string
score                   int
nickname                string
userlevelname           string
userclientshow          string
ismobile                string
days                    int
Time taken: 0.085 seconds, Fetched: 17 row(s)
hive>
```

当然，也可以创建外部表：

view plain copy

create external table if not exists pinglunwb (

productid string,

commentcount int,

goodcount int,

generalcount int,

poorcount int,

goodrateshow float,

generalrateshow float,

poorrateshow float,

guid string,

content string,

creationtime string,

score int,

nickname string,

userclient string,

mobileVersion string,

days int

) row format delimited

fields terminated by '\t'

location '/myedu2/out/1';

外部表的创建方法比内部表多了一个 external，同时还加上了 if 判断，判断创建表之前，是否存在同样名称表。Hive 创建内部表时，会将数据移动到数据仓库指向的路径；创建外部表时，仅记录数据所在的路径， 不对数据的位置做任何改变。在删除表的时候，内部表的元

数据和数据会被一起删除， 而外部表只删除元数据，不删除数据。这样外部表相对来说更加安全些，数据组织也更加灵活，方便共享源数据，生产中常使用外部表。

7.表设计好以后，在 Hive 端使用 load 命令，将/data/edu3 下的 pinglun 导入 Hive 表中。

view plain copy

load data local inpath '/data/edu3/pinglun' into table pinglun;

```
hive>
    > load data local inpath '/data/edu3/pinglun' into table pinglun;
Loading data to table edu3.pinglun
Table edu3.pinglun stats: [numFiles=1, totalSize=358747]
OK
Time taken: 0.706 seconds
hive>
```

在 hive 中，执行查询操作，验证数据是否导入成功。

view plain copy

select * from pinglun limit 10;

```
hadoop@htu202: ~
1003979 fb3a26e638260e8cd0bb1a99fefbb576        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     这款泡面可以，汤头很浓，并且不辣，算是泡面里面的不错的，
家里的老人觉得可以，买牛奶的时候，凑单一起买的，京东买东西真的很方便的说，下单第二天就到了，买的方便，价格也美丽，不错不错不错不错不错呀，可以的，哈哈      2020-07-27 09:11:49
5       定***哒  2               39
1003979 fb80fc689d208ae1fba9ad81992bf8fb        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     当天晚上下的单，第二天中午就到了，不得不佩服京东快递的强
大，在这里给快递小哥点赞！辛苦了！因为平时没事的时候偶尔吃点泡面，在超市购买了两袋，回家一吃，居然上瘾了，味道真心不错，赶着这次活动购买了两箱，味道真是好极了，统一的真心不错！极力推荐
！五星好评！你值得拥有！       2021-10-30 20:53:51     5       V***    4       10.2.2  5
1003979 fba76b9d24c22bc6f2ce68a8dd8451          500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     无限回购，这已经是第N次回购，最喜欢汤达人豚骨面的口味，
喜欢喝它的汤，酸酸辣辣口感，面也很劲道，煮着吃最好不过，还很实惠        2021-10-20 23:55:23     5       锦***里 4       10.2.0  5
1003979 fbb3b45b-11c7-4095-b0f0-4aa816d2fc70    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     这次买了一整箱的干吃面，烧烤味的，拿回家以后打开包裹，感
觉分量挺足的，包装很容易刘打开了，味道好极了，吃到到嘴里脆脆的感觉，家里人都喜欢，吃了还想吃，一下子吃掉3包，碎碎的小面，想起来小时候的方便面，不过自带调料      2020-04-14 11:00:44
5       A***L   2               2
1003979 fbc50f115b14758a7a04d95b8f3c145b        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     家里一直吃这款汤达人的方便面，不油腻，比其他品牌的方便面
口感好，汤汁好喝，吃起来不会很腻，吃惯这款方便面在吃其他品牌的会感觉很腻。       2021-10-17 19:56:20    5奔***牛 4      10.1.6  8
1003979 fbcaf104-792c-4d96-8541-7c3c47f54137    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     汤达人这款泡面拿非常的好吃，它很香，比起康师傅的猪骨豚面
他肯定要好吃很多，很多很多，然后整个面那比一般的泡面要粗一些，摊位呢是非常浓厚的，我现在搅拌了一下就感觉汤白白的，嗯，这个面的味道确实非常非常的好，给大家推荐。       2019-11-23 19:01
:14     5       ****v   2       8.3.4   2
1003979 fbe40c3a4e05d5432ea53b1a8a7db668        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     方便面特别钟爱汤达人，汤好、面好、味道好，喜欢在煮面时放
上大虾，在打入个鸡蛋，那味道真的是绝了！疫情期间要多备点，以备不时之需。       2021-11-09 21:49:35    5闪***魂 2      10.2.2  3
1003979 fc59bb56-b5c6-431c-aaba-74e6ae99aa65    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     包装：是整箱买的，盒子包装的很好，送过来也没有压坏。里面
的面完整，没有压碎。 面饼：面饼比其他家的大一点，味道松脆，有嚼头，口感非常好，是我喜欢的类型！ 情怀：吃的是干脆面，吃的是一种情怀，是儿时美好的回忆！       2020-02-29
10:06:21        5       w***n   2               22
1003979 fc5bb1d782244ed966674484415dcdb2        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     本人觉得汤达人主要是汤好喝，汤味很浓郁，配上面酸酸辣辣的
，很不错，但第一次买袋装的汤达人，不知道跟杯装的有没有区别，吃完再来评价       2021-10-14 18:07:08    5****w  4       10.1.6  1
1003979 fc8697db1a7d30890eea59f33324489b        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     方便面很好吃，这回因为这边的疫情买了一箱子先屯着，平时早
上吃个早饭也可以泡着吃，味道很不错的，我个人还是喜欢绿色的袋子的味道，更传统一些，味道更正感觉  2021-11-07 23:26:40    5       哎***味 4       10.2.2  6
1003979 fcbae208-5d46-4f6d-88a9-b12df4b2b07f    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     吃完来买的，属于老顾客。味道很不错，经常在这里购买，很喜
欢这个汤达人。 基本上各种都味道的都吃过来了，还是这两种味道的吃起来比较不错，以后再买就继续这两张味道的就OK了。       2020-04-16 21:17:34     5       风***乐 4       5
1003979 fcf94ec69d38eaa20a77d0131cb635a2        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     汤达人的泡面很好吃，面很有韧性，我都是直接用小锅来煮，这
样才比较熟一点，买过很多次，汤也很好喝，加青菜加鸡蛋，很美味，比超市便宜一点     2021-10-26 12:16:41    5u***j  4       10.1.6  11
1003979 fd5e6741ea879c1b1d62e3fa7b787cfb        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     比超市便宜一些，一直煮这个汤达人猪骨浓汤口味的，汤鲜味美
，很好吃。煮的时候加一点菜菜，打两个荷包蛋再加一根火腿肠，味道完美～     2021-10-08 21:18:59     5       于***巴 4       10.1.6  12
1003979 fd5ffdb9-7e3e-4296-a38e-a94ae1b9edbc    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     办公室公认最好吃的面之一，尤其汤的味道特别香，浓浓的果然
有点浓汤的感觉，简直是泡面中的佼佼者，这次京东物流也很给力，浓汤配上鸡蛋和青菜，轻轻松松大厨的味道出来，小孩子也可以吃，鲜美可口，方便。而且价格也算公道，好评  2020-03-25 16:51:10
5       b***r   4               59
1003979 fd7d1835-e330-4aea-9802-eb41fbcc9eaf    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     这是我最喜欢的方便面之一，面条劲道，不会像其他面条煮久了
就变成一坨，没有嚼劲。这个不光面条好吃，汤也是最爱，浓浓的一大碗，配上青菜和鸡蛋，比得上好吃的大餐。家里经常备着这款面条。时不时解解馋，还是很好的！     2020-06-01 22:28:13     5****l
2               27
1003979 fdf7aa9c-5e38-4c7e-8d97-2eded165e4e2    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     汤达人非常好，煮的面条很劲道爽滑，真有点日式拉面的感觉，
和泡的杯面不一样。真的是太棒了，也不知道是某师傅徒了还是怎么回事，很近吃不到真的方便面了。这个真的是非常棒，不管是汤和面都一级棒，而且竟然jd配送都没有碎。很棒！       2020-06-19 17:28
:06     5       榴***丶 2               5
1003979 fe0c9b6c-0e15-4b83-bd0f-2c798877838d    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     就是喜欢京东的配送速度，一如既往的神速。这次的小浣熊干脆
面竟然是水浒系列的，不知道我的运气有没那么的好，开几张卡呢。干脆面还是和以前一样，干脆香甜，很可口。特别是调料放的很适中，个人觉得非常的不错，很是推荐喜欢干脆面的小伙伴哟。     2020-04-0
4 18:16:41      5       小***彡 4               8
1003979 fe8377ed-23c2-4740-a77f-4e61d988ad0a    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     自营的产品我一直都是非常喜欢的，主要还是京东物流好，东西
没有破损，5连包，统一的汤达人泡面真的是超赞，特别是这个口味我很喜欢吃，想在网上买又等不及一定要上京东买，次日达的京东大爱了。这款泡面特别好吃，我觉得挺健康的。下次再来！     2019-11-23
 13:39:20       5       江***7  4               8.3.6   1
1003979 ff81e387-76dc-4755-aed0-550e075a05b2    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     酸酸辣辣豚骨面：可以看到有3包配料，按广告的说法是甄选上
等食材，荟萃猪骨、鸡骨、昆布等精华，文火慢炖，精心熬煮大包汤底，配以醇香可口镇江香醋，辅以木耳、枸杞、芝麻、辣椒等丰富蔬菜及三块大大的美味叉烧肉，酸辣爽口。 汤达人这个面真的必须吃好吗
！强烈推荐酸辣味黄色包装！这个面煮的时候加个蛋就太绝了！！！！ 汤也特别好喝！     2020-02-27 01:55:15     5       y***4   4               31
1003979 ff967b7d-d298-443e-8c25-962688da8471    500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     汤达人是我最喜欢吃的方便面，身为初三党，每天学习熬夜，拿
这个当夜宵，非常不错，这个杯面的量对我来说刚刚好，惊喜的是，新加了叉烧，也非常美味，超级推荐！！！       2020-03-02 23:26:53     5       搞***6  4               1
1003979 ffbfb29e672caa9ff49ce9778440a932        500万+  9400+   12695184300+    272万+  9100+   4600+   99.0    1.0     0.0     非常好吃，放一个西红柿，再放上小青菜菠菜，香菜和香肠，那
简直是一觉，非常的酸爽，清爽， 面非常的劲道，吃了之后，再和汤，汤汁回味无穷 今天做了之后忘了拍照，吃完才想起来，太好吃啦      2021-12-02 22:13:48     5       怪***人 4       10.2.4  1
Time taken: 1.125 seconds, Fetched: 1000 row(s)
hive>
```

查看数据条数

view plain copy

select count(1) as num from pinglun;

```
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.68 sec   HDFS Read: 367293 HDFS Write: 4 SUCCESS
Total MapReduce CPU Time Spent: 3 seconds 680 msec
OK
990
Time taken: 27.677 seconds, Fetched: 1 row(s)
```

2.3.2.2 需求分析：

ismobile 分析

1、需求 1：分析用户使用移动端购买还是 PC 端购买，及移动端和 PC 端的用户比例

SQL 语句：

view plain copy

```
select
  case
    when mobileVersion!='' then '1' else '0'
  end as mobileVersion,
  count(1) as num
from pinglun
group by mobileVersion;
```

```
OK
0        570
1        430
```

其中 1 代表移动端购买，共 430 人，0 代表 PC 端购买，共 570 人。
根据此统计数据，得到如下规律：基于采集到的数据，可判断买此方便面使用 PC 端的比移动端的多。但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

days 分析

2、需求 2：分析用户评论周期（收到货后，一般多久进行评论）
SQL 语句：
view plain copy

```
select
 days,
 count(1) as num
from pinglun
group by days
order by num desc;
```

```
OK
1        131
2        92
3        65
4        46
5        46
6        45
9        35
7        32
0        26
8        26
10       22
13       21
15       19
14       19
11       16
12       15
21       13
20       12
39       12
17       12
```

通过分析结果，我们可以清楚地了解到收到货后第一天，评论 131 人，第二天 2 天评论 92 人，3 天评论的有 65 人，等等…根据此统计数据，得到如下规律到货时间与评论量成反比，但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

userlevel 分析

3、需求 3：分析会员级别（判断购买此商品的用户级别）

SQL 语句：

view plain copy

```
select
  userclient,
  count(1) as num
from pinglun
group by userclient
order by num desc;
```

```
OK
4        600
2        385
0        14
29       1
```

通过分析结果，我们可以看到购买用户会员级别为 4 的最多，有 600 人，2 级会员有 385 人，29 级会员 1 人等，根据此统计数据，得到如下规律：不开会员和高级会员的人占少数，开会员但是等级不高的人占大多数。

但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

kwcloud 分析

4.需求 4：分析评论中出现的词汇

SQL 语句：

view plain copy

```
select
  substr(creationtime, 0, 10) as dt,
  count(1) as num
from pinglun
group by substr(creationtime, 0, 10)
order by num desc;
```

kw	num
的	3463
	1676
吃	1563
很	1539
了	1498
是	1270
面	1115
好	966
汤	897
味道	823
也	725
买	712
人	655
就	628
不	615
好吃	594
方便面	557
都	541
还	493
京东	470
非常	468

排除无效词后，594 个‘好吃’和 1115 和 897 个‘面’和‘汤’通过分析得到如下结论：该方便面味道比较好，而且顾客比较关注汤和面的质量

但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

CreationHour 分析

5.需求五：分析评论创建与时间段的关系

```
select
  substr(creationtime, 11, 3) as dt,
  count(1) as num
from pinglun
group by substr(creationtime, 11, 3)
order by num desc;
```

```
OK
 11      72
 21      72
 10      70
 12      64
 19      64
 16      62
 22      61
 18      60
 14      57
 20      56
 09      53
 13      52
 15      51
 17      47
 23      45
 08      33
 00      25
 06      14
 07      13
 01      13
 04      6
 05      5
 02      3
 03      2
```

根据此统计数据，得到如下规律，顾客在凌晨 12 点到 7 点评论量占少数，8 点到 23 点评论量占多数，并且在 11 点和 21 点达到评论的顶峰均为 72 个

但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

comment 分析

6.需求六：分析评论量

```
select
*
from pinglun
group by pinglun
order by num desc;
```

Pinglun	num
poorcount	4600
generalcount	9400
goodcount	2720000

根据此统计数据，得到如下规律

由 272 万的好评和远低于好评的差评得到，该商品的质量相当不错，十分受顾客喜爱

但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

CreationMonth 分析

7.需求七：分析评论创建与月份的关系

```
select
  substr(creationtime, 6, 2) as dt,
  count(1) as num
from pinglun
group by substr(creationtime, 6, 2)
order by num desc;
```

```
OK
11      180
10      172
12      124
09      94
01      73
06      65
04      59
07      53
02      53
08      52
03      50
05      25
```

根据此统计数据，得到如下规律
由此可得而且后半年的评论量占总评论量的多数，前半年占少数
因为买了商品才能够评论，说明该商品后半年的销量比前半年多
但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

CreationYear 分析

8.需求八： 评论创建与年份的关系

```
 substr(creationtime, 0, 4) as dt,
 count(1) as num
from pinglun
group by substr(creationtime, 0, 4)
order by num desc;
```

creationyear	num
2020	515
2021	278
2019	170
2018	15
2017	12
2015	7
2016	2
2014	1

根据此统计数据，得到如下规律
2014 年到 2020 年逐年评论量上升，在 2019 到 2020 的评论增量最多达到了 345 个说明该商品销售量在 2014 到 2020 逐年增加，而在 2019 年到 2020 年增长的最多，应该是与 2019 年肆虐的新冠病毒有关系如大家在家屯货，但在 2020 到 2021 年评论量大幅下降，应该是与 2021 年中国政府治理疫情，疫情得到极大改善，缓解了市民的恐慌有关系。
但要注意，具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

score 分析

9.需求九：商品评分
select
*
from commentsql
order by num desc;

score	num
5	988
3	12

988 个 5 分，12 个三分
根据此统计数据，得到如下规律
该商品的质量相当不错，十分受顾客喜爱

2.4 数据导出

2.4.1 原理：

Sqoop 导入原理：
在导入开始之前，Sqoop 使用 JDBC 来检查将要导入的表。他检索出表中所有的列以及列的 SQL 数据类型。这些 SQL 类型（varchar、integer）被映射到 Java 数据类型（String、Integer

等），在 MapReduce 应用中将使用这些对应的 Java 类型来保存字段的值。Sqoop 的代码生成器使用这些信息来创建对应表的类，用于保存从表中抽取的记录。Sqoop 启动的 MapReduce 作业用到一个 InputFormat，他可以通过 JDBC 从一个数据库表中读取部分内容。Hadoop 提供的 DataDriverDB InputFormat 能为查询结果进行划分传给指定个数的 map 任务。为了获取更好的导入性能，查询会根据一个“划分列”来进行划分。Sqoop 会选择一个合适的列作为划分列（通常是表的主键）。在生成反序列化代码和配置 InputFormat 之后，Sqoop 将作业发送到 MapReduce 集群。Map 任务将执行查询并将 ResultSet 中的数据反序列化到生成类的实例，这些数据要么直接保存在 SequenceFile 文件中，要么在写到 HDFS 之前被转换成分割的文本。Sqoop 不需要每次都导入整张表，用户也可以在查询中加入到 where 子句，以此来限定需要导入的记录。

Sqoop 导出原理：

Sqoop 导出功能的架构与其导入功能非常相似，在执行导出操作之前，Sqoop 会根据数据库连接字符串来选择一个导出方法。一般为 JDBC。然后，Sqoop 会根据目标表的定义生成一个 Java 类。这个生成的类能够从文本文件中解析记录，并能够向表中插入合适类型的值。接着会启动一个 MapReduce 作业，从 HDFS 中读取源数据文件，使用生成的类解析记录，并且执行选定的导出方法。

2.4.2 操作步骤：

以将 ismobilehive 表导入到 ismobilesql 表为例

2.4.2.1.建 hive 表

```
create table ismobilehive(
ismobile string,
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

ismobilehive 表，有两个字段：设备类型及设备数量

```
hive> desc ismobilehive;
OK
ismobile                string
num                     int
Time taken: 0.219 seconds, Fetched: 2 row(s)
hive>
```

2.4.2.2 导入数据到 hive 表

insert into table ismobilehive
select
case
when mobileVersion !='' then 1
else 0
end as ismobile,
count(1) as num
from pinglun
group by ismobile;

2.4.2.3 查看 hive 表

Select * from ismobilehive;

```
hive> select * from ismobilehive;
OK
0       570
1       430
Time taken: 1.326 seconds, Fetched: 2 row(s)
hive>
```

2.4.2.4 创建 MySQL 表

在 mysql 中，创建名为 edu4out 的数据库，用于存储导过来的数据
view plain copy
CREATE DATABASE IF NOT EXISTS edu4out DEFAULT CHARSET utf8 COLLATE utf8_general_ci;
use edu4out
在 Mysql 中创建名为 ismobilesql 的表，用于储蓄 Hive 中 ismobilehive 表的数据。此表也包含两个字段设备类型及设备数量
view plain copy
create table ismobilesql (ismobile varchar(10),num int);

```
mysql> desc ismobilesql;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| ismobile | varchar(10) | YES  |     | NULL    |       |
| num      | int(11)     | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

2.4.2.5 sqoop 导入

sqoop list-databases \

--connect jdbc:mysql://localhost:3306/ \

--username root \

--password strongs

```
zhangyu@34e6afa4cf3b:/data$ sqoop list-databases \
> --connect jdbc:mysql://localhost:3306/ \
> --username root \
> --password strongs
17/07/07 04:01:18 INFO sqoop.Sqoop: Running Sqoop version: 1.4.5-cdh5.4.5
17/07/07 04:01:18 WARN tool.BaseSqoopTool: Setting your password on the command-line is insecure. Consider using -P inst
ead.
17/07/07 04:01:18 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.
information_schema
hive
jdout
mysql
performance_schema
zhangyu@34e6afa4cf3b:/data$
```

2.4.2.6 mysql 查看

Commentsql

```
+---------------+---------+
| Pinglun       | num     |
+---------------+---------+
| poorcount     |    4600 |
| goodcount     | 2720000 |
| generalcount  |    9400 |
+---------------+---------+
```

以下是其他表导出后的结果，sql 脚本在附录的代码最后

Creationhour

creationhour	num
22	61
07	13
11	72
15	51
18	60
14	57
20	56
09	53
13	52
01	13
04	6
05	5
02	3
03	2
21	72
10	70
17	47
23	45
08	33

Creationmonth

creationmonth	num
09	94
08	52
11	180
03	50
05	25
10	172
12	124
01	73
06	65
04	59
07	53
02	53

Creationyearsql

creationyear	num
2017	12
2015	7
2016	2
2014	1
2020	515
2021	278
2019	170
2018	15

Dayssql

day	num
52	5
50	5
45	5
40	5
37	5
36	5
31	5
51	4
55	4
24	3
28	3
41	3
42	3
63	1
65	1
69	1
80	1
1	131
2	92
3	65

Scoresql

score	num
5	988
3	12

Ismobilesql

ismobile	num
0	570
1	430

Userlevel

userlevel	num
2	385
0	14
4	600
29	1

Kwcloud

每	21
脆弱	1
诗	1
其次	4
碳	2
允许	1
暴力	1
光临	1
西	1
罗	5
霸主	1
切	1
解	9
一个遍	1
广告	16
改造	1
干掉	1
起飞	1

2.5 数据可视化

2.5.1 原理

数据 可视化技术 的基本思想，是将数据库中每一个数据项作为单个图元元素表示，大量的数据集构成数据图像，同时将数据的各个属性值以多维数据的形式表示，可以从不同的维度观察数据，从而对数据进行更深入的观察和分析,我们这里使用 springboot+echarts 的方式完成数据可视化

2.5.2 操作步骤

以顾客评论态度占比分析为例

2.5.2.1、创建实体层

Model 层：

1. Comment

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;

@Data
public class Comment {
    private String pinglun;
    private int num;
}
```

2.5.2.2、创建 Dao 层接口

```
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.Comment;
import com.henan.hebi.wangzhilong.model.CreationTime;

import java.util.List;

public interface CommentDao {
```

```
    List<Comment> getCommentList();
}
```

2.5.2.3、创建 Mapper 层

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.CommentDao">
    <select id="getCommentList"
resultType="com.henan.hebi.wangzhilong.model.Comment">
select case  pinglun when 'poorcount' then '差评'
when 'generalcount' then '中评'
else '好评'
end as pinglun,num
from commentsql

</select>
</mapper>
```

2.5.2.4、创建 Service 层接口，

```
package com.henan.hebi.wangzhilong.service;

import com.henan.hebi.wangzhilong.model.Comment;
import org.springframework.stereotype.Service;

import java.util.List;
@Service
```

```
public interface CommentService {
    List<Comment> getCommentList();
}
```

2.5.2.5、实现 service 层接口

```
package com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.CommentDao;
import com.henan.hebi.wangzhilong.model.Comment;
import com.henan.hebi.wangzhilong.service.CommentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
@Service(value = "CommentService" )
public class CommentServiceImpl implements CommentService {

    @Autowired
    private CommentDao commentDao;

    @Override
    public List<Comment> getCommentList() {
        return commentDao.getCommentList();
    }
}
```

2.5.2.6、创建 controller 层

```
package com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.Comment;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.CommentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;
@RestController
public class CommentController {
    @Autowired
    private CommentService CommentService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/Comments")
    public ResponseEntity<JsonResult> getCommentList (){

        JsonResult r = new JsonResult();
        try {
            List<Comment> Comments = CommentService.getCommentList();
            r.setResult(Comments);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
```

```
        }
        return ResponseEntity.ok(r);
    }

}
```

2.5.2.7.创建显示图表的网页

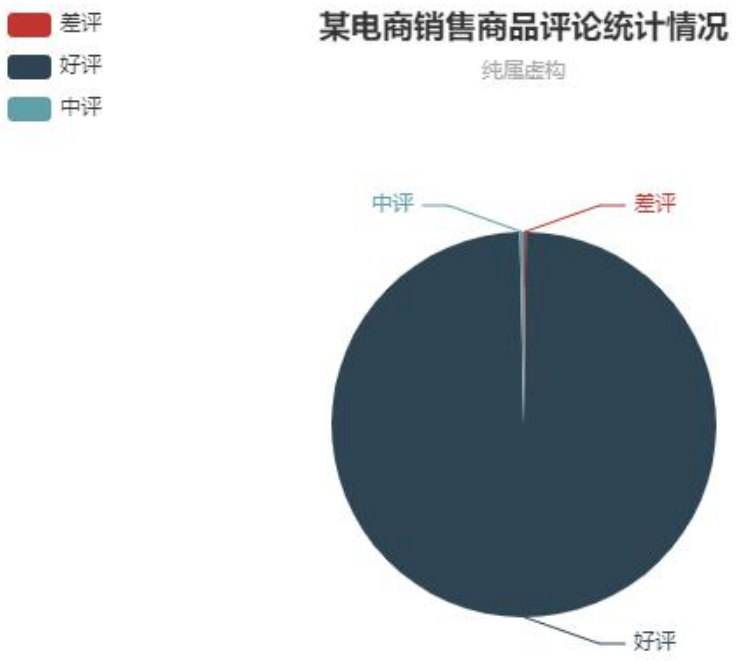
```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        var option = {
            title : {
                text: '某电商销售商品评论统计情况',
                subtext: '',
                x:'center'
```

```
            },
            tooltip : {
                trigger: 'item',
                formatter: "{a} <br/>{b} : {c} ({d}%)"
            },
            legend: {
                orient: 'vertical',
                left: 'left',
            },
            series : [
                {
                    name: '访问来源',
                    type: 'pie',
                    radius : '55%',
                    center: ['50%', '60%'],
                    data:(function () {
                        var datas = [];
                        $.ajax({
                            type:"GET",
                            url:"/Comments",
                            dataType:"json",
                            async:false,
                            success:function (data) {
                                var isMobilesList = data.result;
                                console.log(isMobilesList);
                                for (var i = 0; i < isMobilesList.length;
i++){
                                    datas.push({
                                        "value":isMobilesList[i].num,
"name":isMobilesList[i].pinglun
                                    })
                                }
```

```
                        }
                    });
                    return datas;
                })(),
                itemStyle: {
                    emphasis: {
                        shadowBlur: 10,
                        shadowOffsetX: 0,
                        shadowColor: 'rgba(0, 0, 0, 0.5)'
                    }
                }
            }
        ]
    };
    myChart.setOption(option);
</script>
</div>
</body>
</html>
```

2.5.2.8.数据可视化的界面分析

运行结果：

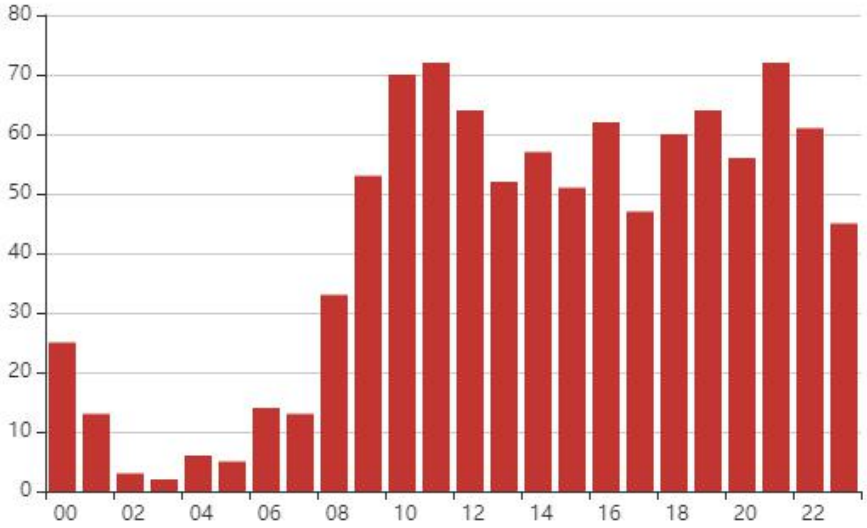


结论：根据此统计数据，得到如下规律
由 272 万的好评和远低于好评的差评得到，该商品的质量相当不错，十分受顾客喜爱

2.5.2.9、其它需求分析结果

CreationHour 页面

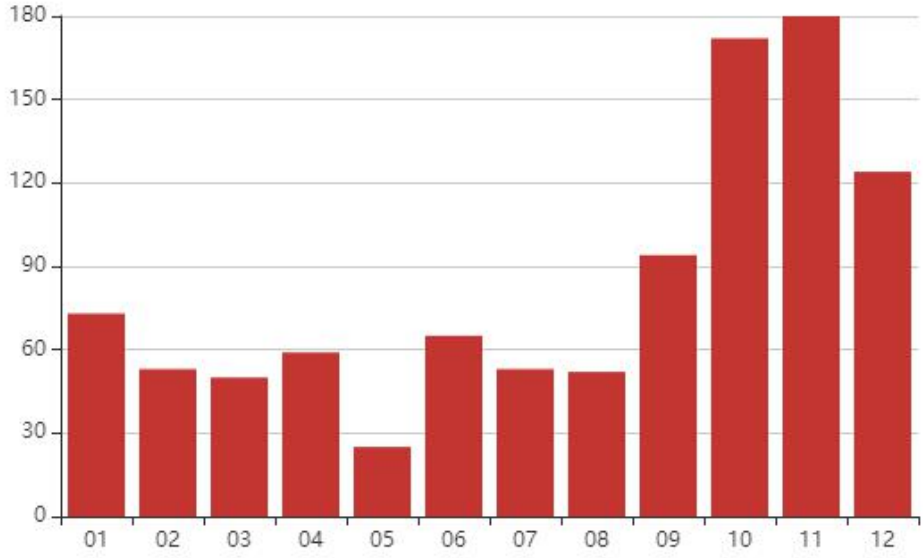
评论创建与时间段的关系



结论:根据此统计数据，得到如下规律，顾客在凌晨 12 点到 7 点评论量占少数，8 点到 23 点评论量占多数，并且在 11 点和 21 点达到评论的顶峰均为 72 个

CreationMonth 页面

评论创建与月份的关系



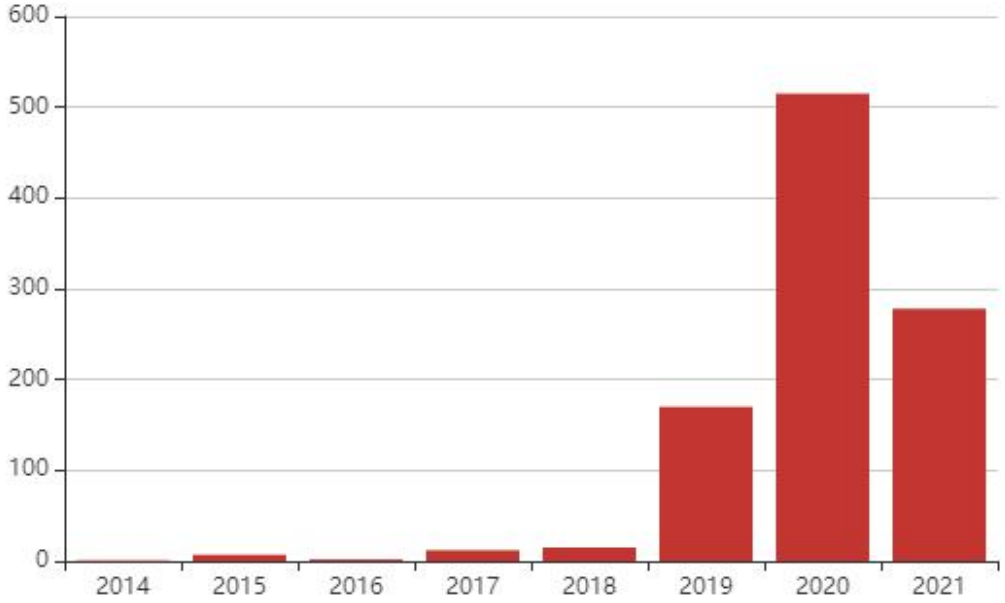
结论:根据此统计数据，得到如下规律

由此可得 12 月份到 5 月份该评论量随时间成反比，6 月份到 11 月份成正比

而且后半年的评论量占总评论量的多数，前半年占少数

CreationTime 页面

评论创建年份



结论:

根据此统计数据，得到如下规律

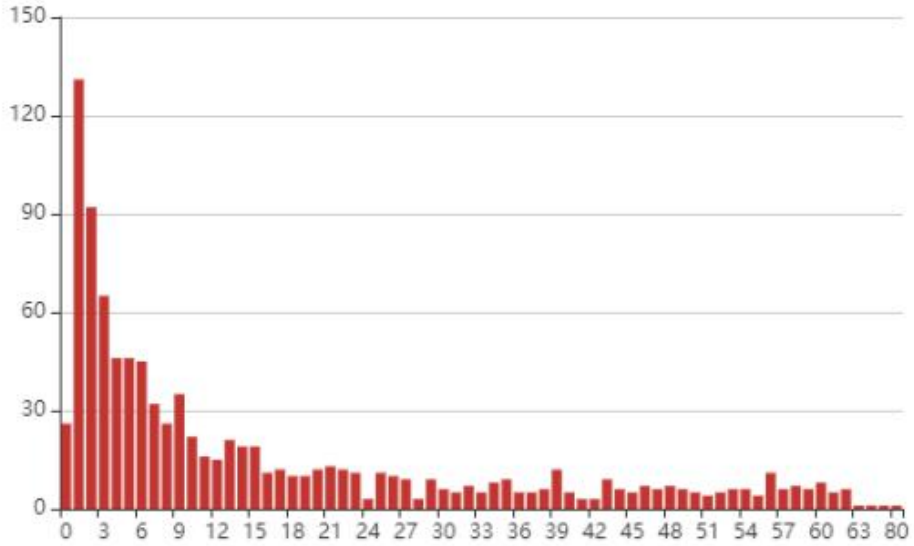
2014 年到 2020 年逐年评论量上升，在 2019 到 2020 的评论增量最多达到了 345 个

说明该商品销售量在 2014 到 2020 逐年增加，而在 2019 年到 2020 年增长的最多，

应该是与 2019 年肆虐的新冠病毒有关系，但在 2020 到 2021 年评论量大幅下降，应该是与 2021 年中国政府治理疫情，是的疫情得到极大改善有关系.

Days 页面

到货几天后评论人数

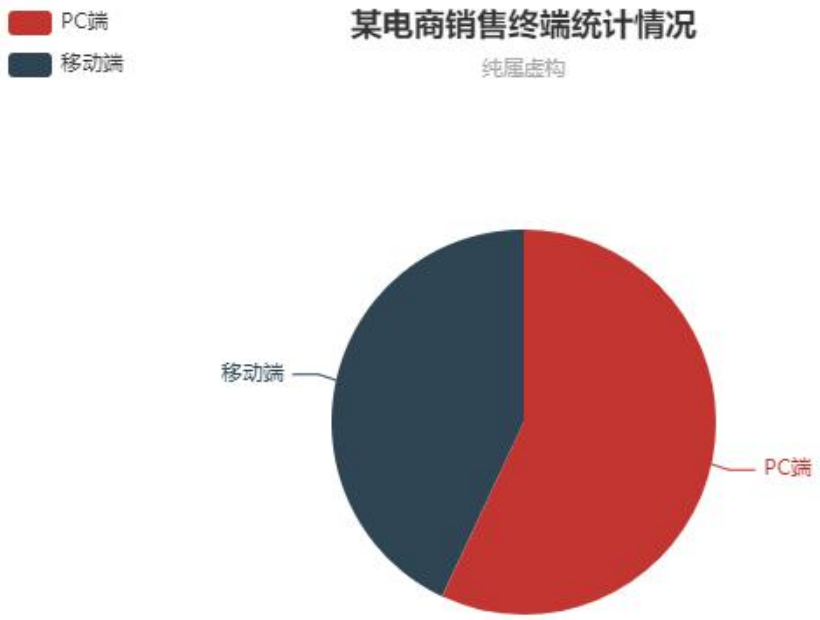


结论

通过分析结果，我们可以清楚地了解到收到货后第一天，评论 131 人，第二天 2 天评论 92 人，3 天评论的有 65 人，等等…根据此统计数据，得到如下规律

到货时间与评论量成反比

Ismobile 页面



结论

其中 1 代表移动端购买，共 430 人，0 代表 PC 端购买，共 570 人。根据此统计数据，得到如下规律：买此方便面使用 PC 端的比移动端的多

KwCloud 页面

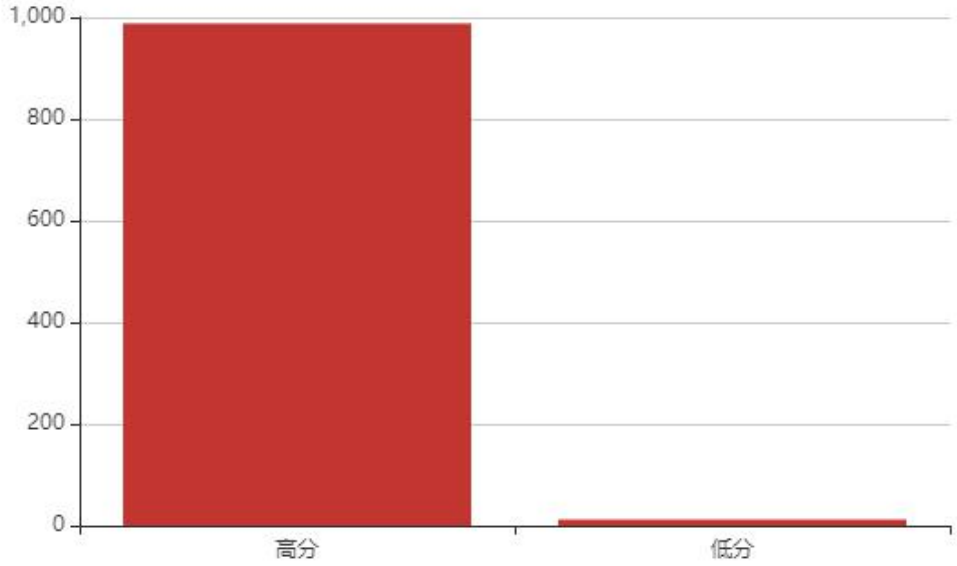
某方便面评论词云

结论:排除无效词后，594 个‘好吃’和 1115 和 897 个‘面’和‘汤’通过分析得到

如下结论：该方便面味道比较好，而且顾客比较关注汤和面的质量

Score 页面

商品评分

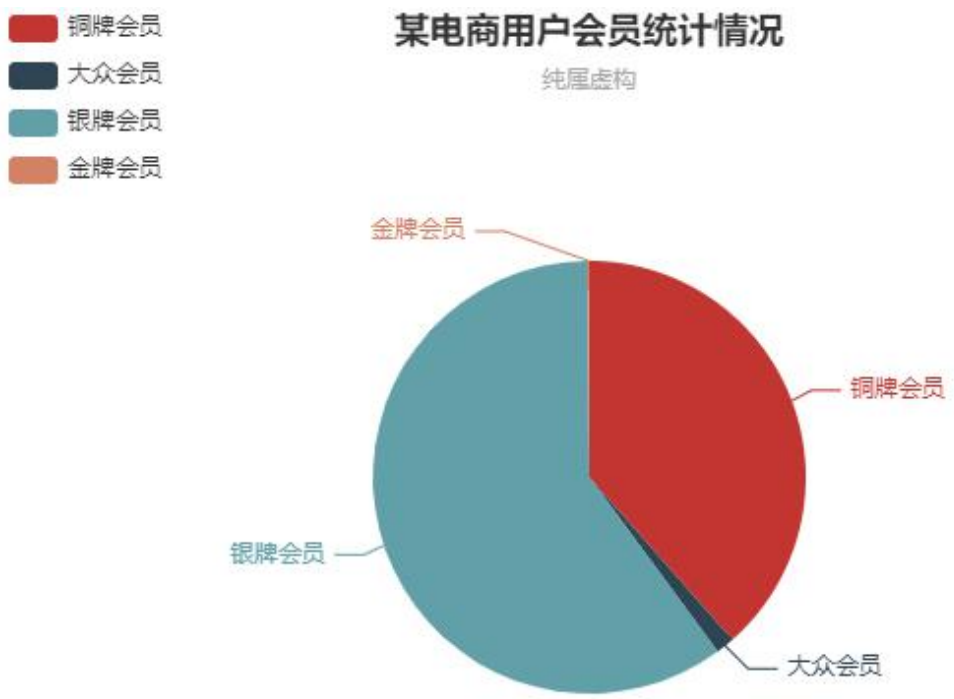


结论:988 个 5 分，12 个三分

根据此统计数据，得到如下规律

该商品的质量相当不错，十分受顾客喜爱

Userlevel 页面



结论:通过分析结果，我们可以看到用户会员级别为银牌的最多，占一半以上，铜牌会员占大概 40%，金牌会员最少等，根据此统计数据，得到如下规律

不开会员和高级会员的人占少数，开会员但是等级不高的人占大多数

具体数值，会根据采集来的数据的变化而变化。时间和商品不同，会导致数值有波动。

3、项目总结

总结通过整个项目实施有哪些收获和认知？哪些方面有怎样的提高？

在老师带领下完成了京东评论分析的项目。

以下是我在该项目实施各个阶段的收获

数据采集阶段：了解了数据采集的原理，掌握了数据采集的操作步骤，选择需要采集的内容，对网站代码进行分析，获取我们用于爬取的 URL，编写爬虫代码进行爬取，查看爬取到的结果，并且基本掌握了 java 爬虫的使用方法，能够自主的爬取一些需要的数据比那个且在这个过程中，对 JSON 数据格式有了更加深刻的理解，JSON 数据是网路上传输数据常用的格式。

数据清洗阶段：了解了数据数据清洗的含义，数据清洗就是剔除脏数据，获取我们所需要的数据，掌握了数据清洗的基本步骤，选取所需字段,搭建数据清洗解析框架，编写数据清洗代码，查看数据清洗结果

数据分析阶段：了解了数据分析的原理，通过对大量数据的观察，总结出规律，基本上掌握了数据分析的步骤：准备好数据分析所需要的环境，导入索要分析的总数据并建表，创建数据分析表进行分析。

数据导出阶段：了解数据导出的原理，就是使用 sqoop 将数据从 hdfs 中导入到 mysql 中，基本掌握了数据导出的步骤：数据导出准备，在 MySQL 中建表创建输入，使用 sqoop 将数据从 hdfs 中导入到 mysql

中，在 mysql 中查看数据是否导入成功。

数据可视化阶段：了解了数据可视化的原理，为了清晰有效地传递信息，数据可视化使用信息图表和其他工具。可以使用点、线或条对数字数据进行编码，以便在视觉上传达定量信息。[2] 有效的可视化可以帮助用户分析和推理数据和证据。它使复杂的数据更容易理解和使用。用户可能有特定的分析任务（如进行比较或理解因果关系），以及该任务要遵循的图形设计原则。表格通常用于用户查找特定的度量，而各种类型的图表用于显示一个或多个变量的数据中的模式或关系。大致掌握了数据可视化的步骤，在 model 层创建实体类，在 Dao 层创建接口，在 mapper 层实现 Dao 层接口，在 service 层创建服务接口与服务实现，在 controller 层创建 controller 类，在 resources 层的 template 层创建用于显示图表的网页，最后在 Controller 层创建返回该所有网页的 controller 层。

4 附录（项目源码）

model

2. CreationHour

```
package com.henan.hebi.wangzhilong.model;


import lombok.Data;


@Data
public class CreationHour {
    private String creationhour;
    private int num;
}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;


import com.henan.hebi.wangzhilong.model.CreationHour;
import com.henan.hebi.wangzhilong.model.CreationTime;


import java.util.List;


public interface CreationHourDao {
    List<CreationHour> getCreationHourList();
}
```

Mapper

```
<?xml version="1.0" encoding="utf-8" ?>

<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.CreationHourDao">
    <select id="getCreationHourList"
resultType="com.henan.hebi.wangzhilong.model.CreationHour">
select * from creationhour order by creationhour


</select>
</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;


import com.henan.hebi.wangzhilong.model.CreationHour;


import java.util.List;


public interface CreationHourService {
    List<CreationHour> getCreationHourList();


}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.CreationHourDao;
import com.henan.hebi.wangzhilong.model.CreationHour;
import com.henan.hebi.wangzhilong.service.CreationHourService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
@Service(value = "CreationHourService" )
public class CreationHourServiceImpl implements CreationHourService {
    @Autowired
    private CreationHourDao CreationHourDao;

    @Override
    public List<CreationHour> getCreationHourList() {
        return CreationHourDao.getCreationHourList();
    }

}
```

Controller

```
package com.henan.hebi.wangzhilong.controller;


import com.henan.hebi.wangzhilong.model.CreationHour;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.CreationHourService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
@RestController

public class CreationHourController {
    @Autowired
    private CreationHourService CreationHourService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/CreationHours")
    public ResponseEntity<JsonResult> getCreationHourList (){

        JsonResult r = new JsonResult();
        try {
            List<CreationHour> CreationHours =
CreationHourService.getCreationHourList();
            r.setResult(CreationHours);
            r.setStatus("ok");
        } catch (Exception e) {
```

```
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }

}
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div>
    <h3>评论创建时间段</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
```

```
// 新建 daysList 与 numList 数组来接受数据
var daysList = [];
var numList = [];
$.ajax({
    type:"GET",
    url:"/CreationHours",
    dataType:"json",
    async:false,
    success:function (data) {
        var dataList = data.result;
        console.log(dataList);
        for (var i = 0; i < dataList.length; i++){
            daysList.push(dataList[i].creationhour);
            numList.push(dataList[i].num);
        }
        console.log(daysList);
        console.log(numList);
    },
    error :function(errorMsg) {
        alert("获取后台数据失败！");
    }
});
option = {



    xAxis: {
        type: 'category',
        data:daysList
    },
    yAxis: {
        type: 'value'
    },
```

```
            series: [
                {
                    data:numList,
                    type: 'bar',
                    showBackground: true,
                    backgroundStyle: {
                        color: 'rgba(180, 180, 180, 0.2)'
                    }
                }
            ]
        };
        myChart.setOption(option);



    </script>
</div>

</body>
</html>
```

3. CreationMonth

Model

```
package com.henan.hebi.wangzhilong.model;


import lombok.Data;


@Data
public class CreationMonth {
```

```
    private String creationmonth;
    private int num;

}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;


import com.henan.hebi.wangzhilong.model.CreationMonth;


import java.util.List;


public interface CreationMonthDao {
    List<CreationMonth> getCreationMonthList();
}
```

Mapper

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.CreationMonthDao">
    <select id="getCreationMonthList"
resultType="com.henan.hebi.wangzhilong.model.CreationMonth">
select * from creationmonth order by creationmonth


</select>
</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;


import com.henan.hebi.wangzhilong.model.CreationMonth;


import java.util.List;


public interface CreationMonthService {
    List<CreationMonth> getCreationMonthList();
}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;


import com.henan.hebi.wangzhilong.dao.CreationMonthDao;
import com.henan.hebi.wangzhilong.model.CreationMonth;
import com.henan.hebi.wangzhilong.service.CreationMonthService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import java.util.List;
@Service(value = "CreationMonthService" )
public class CreationMonthServiceImpl implements CreationMonthService {
    @Autowired
    private CreationMonthDao CreationMonthDao;

    @Override
    public List<CreationMonth> getCreationMonthList() {
        return CreationMonthDao.getCreationMonthList();
    }

}
```

Controller

```
package com.henan.hebi.wangzhilong.controller;


import com.henan.hebi.wangzhilong.model.CreationMonth;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.CreationMonthService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
```

```
@RestController
public class CreationMonthController {
    @Autowired
    private CreationMonthService CreationMonthService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/CreationMonths")
    public ResponseEntity<JsonResult> getCreationMonthList (){

        JsonResult r = new JsonResult();
        try {
            List<CreationMonth> CreationMonths =
CreationMonthService.getCreationMonthList();
            r.setResult(CreationMonths);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }

}
```

html

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div>
    <h3>评论创建月份</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        // 新建 daysList 与 numList 数组来接受数据
        var daysList = [];
        var numList = [];
        $.ajax({
            type:"GET",
            url:"/CreationMonths",
            dataType:"json",
            async:false,
            success:function (data) {
                var dataList = data.result;
                console.log(dataList);
                for (var i = 0; i < dataList.length; i++){
                    daysList.push(dataList[i].creationmonth);
                    numList.push(dataList[i].num);
```

```
            }
            console.log(daysList);
            console.log(numList);
        },
        error :function(errorMsg) {
            alert("获取后台数据失败！");
        }
    });
    option = {



        xAxis: {
            type: 'category',
            data:daysList
        },
        yAxis: {
            type: 'value'
        },
        series: [
            {
                data:numList,
                type: 'bar',
                showBackground: true,
                backgroundStyle: {
                    color: 'rgba(180, 180, 180, 0.2)'
                }
            }
        ]
    };
    myChart.setOption(option);
```

```
    </script>
</div>

</body>
</html>
```

Model

4. CreationTime

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;

@Data
public class CreationTime {
    private String creationyear;
    private int num;
}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.CreationTime;

import java.util.List;

public interface CreationTimeDao {

    List<CreationTime> getCreationTimeList();
}
```

Mapper

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.CreationTimeDao">
    <select id="getCreationTimeList"
resultType="com.henan.hebi.wangzhilong.model.CreationTime">
select * from creationyearsql order by creationyear

    </select>
</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;


import com.henan.hebi.wangzhilong.model.CreationTime;


import java.util.List;


public interface CreationTimeService {
    List<CreationTime> getCreationTimeList();
}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;


import com.henan.hebi.wangzhilong.dao.CreationTimeDao;
import com.henan.hebi.wangzhilong.dao.IsMobileDao;
import com.henan.hebi.wangzhilong.model.CreationTime;
import com.henan.hebi.wangzhilong.service.CreationTimeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
@Service(value = "CreationTimeService" )
public class CreationTimeServiceImpl implements CreationTimeService {
    @Autowired
```

```
    private CreationTimeDao creationTimeDao;

    @Override
    public List<CreationTime> getCreationTimeList() {
        return creationTimeDao.getCreationTimeList();
    }
}
```

Controller

```
package com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.CreationTime;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.CreationTimeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
public class CreationTimeController {
```

```
    @Autowired
    private CreationTimeService CreationTimeService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/CreationYears")
    public ResponseEntity<JsonResult> getCreationTimeList (){

        JsonResult r = new JsonResult();
        try {
            List<CreationTime> CreationTimes =
CreationTimeService.getCreationTimeList();
            r.setResult(CreationTimes);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }
}
```

```
html
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div>
    <h3>评论创建年份</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        // 新建 daysList 与 numList 数组来接受数据
        var daysList = [];
        var numList = [];
        $.ajax({
            type:"GET",
            url:"/CreationYears",
            dataType:"json",
            async:false,
            success:function (data) {
                var dataList = data.result;
                console.log(dataList);
                for (var i = 0; i < dataList.length; i++){
                    daysList.push(dataList[i].creationyear);
                    numList.push(dataList[i].num);
                }
                console.log(daysList);
```

```
                console.log(numList);
            },
            error :function(errorMsg) {
                alert("获取后台数据失败！");
            }
        });
        option = {



            xAxis: {
                type: 'category',
                data:daysList
            },
            yAxis: {
                type: 'value'
            },
            series: [
                {
                    data:numList,
                    type: 'bar',
                    showBackground: true,
                    backgroundStyle: {
                        color: 'rgba(180, 180, 180, 0.2)'
                    }
                }
            ]
        };
        myChart.setOption(option);



    </script>
</div>
```

```
</body>
</html>
```

Days

Model

```
package com.henan.hebi.wangzhilong.model;


import lombok.Data;


@Data
public class Days {
private int day;
private int num;
```

```
}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.Days;

import java.util.List;

public interface DaysDao {
    List<Days> getDaysList();
}
```

Mapper

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.DaysDao">
<select id="getDaysList"
resultType="com.henan.hebi.wangzhilong.model.Days">
select * from dayssql order by day

</select>
</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;


import com.henan.hebi.wangzhilong.model.Days;


import java.util.List;


public interface DaysService {
    List<Days> getDaysList();
}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;


import com.henan.hebi.wangzhilong.dao.DaysDao;
import com.henan.hebi.wangzhilong.dao.IsMobileDao;
import com.henan.hebi.wangzhilong.model.Days;
import com.henan.hebi.wangzhilong.model.IsMobile;
import com.henan.hebi.wangzhilong.service.DaysService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;


import java.util.List;
@Service(value = "DaysService" )
public class DaysServiceImpl implements DaysService {
    @Autowired
    private DaysDao daysDao;


    @Override
```

```
    public List<Days> getDaysList() {
        return daysDao.getDaysList();
    }
}
```

```
Controller
package com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.Days;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.DaysService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class DaysController {

    @Autowired
    private DaysService DaysService;
```

```
    /**
     * 查询列表
     */
    @GetMapping(value = "/Dayss")
    public ResponseEntity<JsonResult> getDaysList (){

        JsonResult r = new JsonResult();
        try {
            List<Days> Dayss = DaysService.getDaysList();
            r.setResult(Dayss);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }
}
```

```
html
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
```

```
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div>
    <h3>到货几天后评论人数</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        // 新建 daysList 与 numList 数组来接受数据
        var daysList = [];
        var numList = [];
        $.ajax({
            type:"GET",
            url:"/Dayss",
            dataType:"json",
            async:false,
            success:function (data) {
                var dataList = data.result;
                console.log(dataList);
                for (var i = 0; i < dataList.length; i++){
                    daysList.push(dataList[i].day);
                    numList.push(dataList[i].num);
                }
                console.log(daysList);
                console.log(numList);
            },
            error :function(errorMsg) {
                alert("获取后台数据失败！");
            }
```

```
        });
        option = {
            xAxis: {
                type: 'category',
                data:daysList
            },
            yAxis: {
                type: 'value'
            },
            series: [
                {
                    data:numList,
                    type: 'bar',
                    showBackground: true,
                    backgroundStyle: {
                        color: 'rgba(180, 180, 180, 0.2)'
                    }
                }
            ]
        };
        myChart.setOption(option);



    </script>
</div>

</body>
</html>
```

IsMobile

Model

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;
@Data
public class IsMobile {
    private String ismobile;
    private int num;
}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.IsMobile;

import java.util.List;
```

```
public interface IsMobileDao {

List<IsMobile> getIsMobileList();}
```

```
Mapper
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.IsMobileDao">
    <select id="getIsMobileList"
resultType="com.henan.hebi.wangzhilong.model.IsMobile">
        select case ismobile when '0' then 'PC端' else '移动端' end as
ismobile,num  from ismobilesql
    </select>
</mapper>
```

```
Service
package com.henan.hebi.wangzhilong.service;

import com.henan.hebi.wangzhilong.model.IsMobile;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;
```

```
public interface IsMobileService {

    List<IsMobile> getIsMobileList();



}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.IsMobileDao;
import com.henan.hebi.wangzhilong.model.IsMobile;
import com.henan.hebi.wangzhilong.service.IsMobileService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service(value = "isMobileService" )
public class IsMobileServiceImpl implements IsMobileService {
   @Autowired
   private IsMobileDao isMobileDao;

    @Override
    public List<IsMobile> getIsMobileList() {
        return isMobileDao.getIsMobileList();
    }

}
```

Controller

```
package com.henan.hebi.wangzhilong.controller;
import com.henan.hebi.wangzhilong.model.IsMobile;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.IsMobileService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Repository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class IsMobileController {
    @Autowired
    private IsMobileService isMobileService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/ismobiles")
    public ResponseEntity<JsonResult> getIsMobileList (){

        JsonResult r = new JsonResult();
        try {
            List<IsMobile> ismobiles =
isMobileService.getIsMobileList();
```

```
            r.setResult(ismobiles);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }



}
```

```
html
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>京东评论终端占比</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 600px;height:400px; float: left">
   <script type="text/javascript">
       var myChart = echarts.init(document.getElementById('main'));
       var option = {
           title : {
               text: '某电商销售终端统计情况',
```

```
                subtext: '纯属虚构',
                x:'center'
            },
            tooltip : {
                trigger: 'item',
                formatter: "{a} <br/>{b} : {c} ({d}%)"
            },
            legend: {
                orient: 'vertical',
                left: 'left',
            },
            series : [
                {
                    name: '访问来源',
                    type: 'pie',
                    radius : '55%',
                    center: ['50%', '60%'],
                    data:(function () {
                        var datas = [];
                        $.ajax({
                            type:"GET",
                            url:"/ismobiles",
                            dataType:"json",
                            async:false,
                            success:function (data) {
                                var isMobilesList = data.result;
                                console.log(isMobilesList);
                                for (var i = 0; i < isMobilesList.length;
i++){
                                    datas.push({
                                        "value":isMobilesList[i].num,
"name":isMobilesList[i].ismobile
```

```
                                })
                            }
                        }
                    });
                    return datas;
                })(),
                itemStyle: {
                    emphasis: {
                        shadowBlur: 10,
                        shadowOffsetX: 0,
                        shadowColor: 'rgba(0, 0, 0, 0.5)'
                    }
                }
            }
        ]
    };
    myChart.setOption(option);
</script>
</div>
</body>
</html>
```

KwCloud

Model

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;

@Data
public class KwCloud {
    private String kw;
    private int num;
}
```

Dao

```
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.KwCloud;

import java.util.List;

public interface KwCloudDao {
    List<KwCloud> getKwCloudList();
}
```

Mapper

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.KwCloudDao">
    <select id="getKwCloudList"
resultType="com.henan.hebi.wangzhilong.model.KwCloud">
select * from kwcloud  where kw not in('是','的','也','京东','很',
```

```
'最非常','就','?','这','都','我','(','了','买','到','这个','最
','d','q','n','0','1'
,'2','3','4','5','6','7','8','9','10','1.','2.','3.','4.')

    </select>

</mapper>
```

```
Service
package com.henan.hebi.wangzhilong.service;

import com.henan.hebi.wangzhilong.model.KwCloud;

import java.util.List;

public interface KwCloudService {
    List<KwCloud> getKwCloudList();
}
```

Serviceimpl

```
ackage com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.DaysDao;
import com.henan.hebi.wangzhilong.dao.KwCloudDao;
import com.henan.hebi.wangzhilong.model.Days;
import com.henan.hebi.wangzhilong.model.KwCloud;
import com.henan.hebi.wangzhilong.service.KwCloudService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service(value = "KwCloudService" )
public class KwCloudServiceImpl implements KwCloudService {
    @Autowired
    private KwCloudDao kwCloudDao;

    @Override
    public List<KwCloud> getKwCloudList() {
        return kwCloudDao.getKwCloudList();
    }
}
```

Controller

```
ppackage com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.KwCloud;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.KwCloudService;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class KwCloudController {

    @Autowired
    private KwCloudService KwCloudService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/KwClouds")
    public ResponseEntity<JsonResult> getKwCloudList (){

        JsonResult r = new JsonResult();
        try {
            List<KwCloud> KwClouds = KwCloudService.getKwCloudList();
            r.setResult(KwClouds);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }
}
```

```
html
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>电商某方便面词云</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
    <script src="JSFile/echarts-wordcloud.min.js"></script>

</head>
<body>
<div>
    <h3>某方便面评论词云</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
<script type="text/javascript">
    // 基于准备好的dom，初始化echarts实例
    var myChart = echarts.init(document.getElementById('main'));

    var datas = [];

    //AJAX接收数据主体
    $.ajax({
        type:"GET",
        url:"/KwClouds",
        dataType:"json",
        async:false,
```

```
        success:function (data) {
            //kwdata = data.detailMsg.data;
        var KwCloudList=data.result;
            for (var i = 0; i <  KwCloudList.length; i++){
                datas.push({
                    "value": KwCloudList[i].num, "name":
KwCloudList[i].kw
                })
            }
        },
        error :function(errorMsg) {
            alert("获取后台数据失败！");
        }
    });

    // 指定图表的配置项和数据
    var option = {
        tooltip: {},
        series: [{
            name:'频率',
            type : 'wordCloud',  //类型为字符云
            shape:'smooth',  //平滑
            gridSize : 2, //网格尺寸
            size : ['80%','80%'],
            sizeRange : [ 35, 100 ],
            rotationRange : [ -50, 80 ], //旋转范围
            textStyle : {
                normal : {
                    fontFamily:'sans-serif',
                    color : function() {
                        return 'rgb('
                            + [ Math.round(Math.random() * 160),
```

```
                                Math.round(Math.random() * 160),
                                Math.round(Math.random() * 160) ]
                                .join(',') + ')';
                    }
                },
                emphasis : {
                    shadowBlur : 5,  //阴影距离
                    shadowColor : '#333'  //阴影颜色
                }
            },
            data: datas
        }]
    };

    // 使用刚指定的配置项和数据显示图表。
    myChart.setOption(option);
</script>
  </div>
</body>
</html>
```

Model

9.Score

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;
```

```
@Data
public class Score {
    private String score;
    private int num;

}
```

```
Dao
package com.henan.hebi.wangzhilong.dao;

import com.henan.hebi.wangzhilong.model.Score;

import java.util.List;

public interface ScoreDao {
    List<Score> getScoreList();
}
```

```
Mapper
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.ScoreDao">
    <select id="getScoreList"
resultType="com.henan.hebi.wangzhilong.model.Score">
 select case score when '5' then '高分' else '低分' end as score,num  from
scoresql

</select>
</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;

import com.henan.hebi.wangzhilong.model.Score;

import java.util.List;

public interface ScoreService {
    List<Score> getScoreList();
}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.ScoreDao;
import com.henan.hebi.wangzhilong.model.Score;
import com.henan.hebi.wangzhilong.service.ScoreService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
@Service(value = "CreationScoreService" )
```

```
public class ScoreServiceImpl implements ScoreService {
    @Autowired
    private ScoreDao ScoreDao;

    @Override
    public List<Score> getScoreList() {
        return ScoreDao.getScoreList();
    }



}
```

```
Controller
package com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.Score;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.ScoreService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import java.util.List;
@RestController
public class ScoreController {
    @Autowired
    private ScoreService ScoreService;

    /**
     * 查询列表
     */
    @GetMapping(value = "/Scores")
    public ResponseEntity<JsonResult> getScoreList (){

        JsonResult r = new JsonResult();
        try {
            List<Score> Scores = ScoreService.getScoreList();
            r.setResult(Scores);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }

}
```

```
html
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>Title</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
<body>
<div>
    <h3>商品评分</h3>
</div>
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        // 新建 daysList 与 numList 数组来接受数据
        var daysList = [];
        var numList = [];
        $.ajax({
            type:"GET",
            url:"/Scores",
            dataType:"json",
            async:false,
            success:function (data) {
                var dataList = data.result;
                console.log(dataList);
                for (var i = 0; i < dataList.length; i++){
                    daysList.push(dataList[i].score);
                    numList.push(dataList[i].num);
                }
                console.log(daysList);
                console.log(numList);
```

```
            },
            error :function(errorMsg) {
                alert("获取后台数据失败！");
            }
        });
        option = {



            xAxis: {
                type: 'category',
                data:daysList
            },
            yAxis: {
                type: 'value'
            },
            series: [
                {
                    data:numList,
                    type: 'bar',
                    showBackground: true,
                    backgroundStyle: {
                        color: 'rgba(180, 180, 180, 0.2)'
                    }
                }
            ]
        };
        myChart.setOption(option);



    </script>
</div>
```

```
</body>
</html>
```

```
Userlevel
Model
package com.henan.hebi.wangzhilong.model;

import lombok.Data;

@Data
public class UserLevel {
private String userlevel;
private int num;
}
```

```
Dao
package com.henan.hebi.wangzhilong.dao;
```

```
import com.henan.hebi.wangzhilong.model.UserLevel;

import java.util.List;

public interface UserLevelDao {
    List<UserLevel> getUserLevelList();
}
```

```
Mapper
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.henan.hebi.wangzhilong.dao.UserLevelDao">
    <select id="getUserLevelList"
resultType="com.henan.hebi.wangzhilong.model.UserLevel">
select case userLevel
when '0' then '大众会员'
when '2' then '铜牌会员'
when '4' then '银牌会员'
else '金牌会员' end as userlevel,num  from userlevelsql

    </select>

</mapper>
```

Service

```
package com.henan.hebi.wangzhilong.service;

import com.henan.hebi.wangzhilong.model.UserLevel;

import java.util.List;

public interface UserLevelService {
    List<UserLevel> getUserLevelList();
}
```

Serviceimpl

```
package com.henan.hebi.wangzhilong.service.impl;

import com.henan.hebi.wangzhilong.dao.DaysDao;
import com.henan.hebi.wangzhilong.dao.UserLevelDao;
import com.henan.hebi.wangzhilong.model.Days;
import com.henan.hebi.wangzhilong.model.UserLevel;
import com.henan.hebi.wangzhilong.service.UserLevelService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service(value = "UserLevelService" )
public class UserLevelServiceImpl implements UserLevelService {
    @Autowired
    private UserLevelDao userLevelDao;

    @Override
    public List<UserLevel> getUserLevelList() {
```

```
        return userLevelDao.getUserLevelList();
    }

}
```

```
Controller
package com.henan.hebi.wangzhilong.controller;

import com.henan.hebi.wangzhilong.model.UserLevel;
import com.henan.hebi.wangzhilong.model.JsonResult;
import com.henan.hebi.wangzhilong.service.UserLevelService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class UserLevelController {

    @Autowired
    private UserLevelService UserLevelService;

    /**
     * 查询列表
     */
```

```
    @GetMapping(value = "/UserLevels")
    public ResponseEntity<JsonResult> getUserLevelList (){

        JsonResult r = new JsonResult();
        try {
            List<UserLevel> UserLevels =
UserLevelService.getUserLevelList();
            r.setResult(UserLevels);
            r.setStatus("ok");
        } catch (Exception e) {
            r.setResult(e.getClass().getName() + ":" + e.getMessage());
            r.setStatus("error");
            e.printStackTrace();
        }
        return ResponseEntity.ok(r);
    }
}
```

```
Html


<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <title>某电商用户会员统计情况</title>
    <script src="JSFile/echarts.min.js"></script>
    <script src="JSFile/jquery-3.5.0.js"></script>
</head>
```

```
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 600px;height:400px; float: left">
    <script type="text/javascript">
        var myChart = echarts.init(document.getElementById('main'));
        var option = {
            title : {
                text: '某电商用户会员统计情况',
                subtext: '纯属虚构',
                x:'center'
            },
            tooltip : {
                trigger: 'item',
                formatter: "{a} <br/>{b} : {c} ({d}%)"
            },
            legend: {
                orient: 'vertical',
                left: 'left',
            },
            series : [
                {
                    name: '会员等级',
                    type: 'pie',
                    radius : '55%',
                    center: ['50%', '60%'],
                    data:(function () {
                        var datas = [];
                        $.ajax({
                            type:"GET",
                            url:"/UserLevels",
                            dataType:"json",
                            async:false,
```

```
                            success:function (data) {
                                var userLevelsList = data.result;
                                console.log(userLevelsList);
                                for (var i = 0; i <
userLevelsList.length; i++){
                                    datas.push({

"value":userLevelsList[i].num, "name":userLevelsList[i].userlevel
                                    })
                                }
                            }
                        });
                        return datas;
                    })(),
                    itemStyle: {
                        emphasis: {
                            shadowBlur: 10,
                            shadowOffsetX: 0,
                            shadowColor: 'rgba(0, 0, 0, 0.5)'
                        }
                    }
                }
            ]
        };
        myChart.setOption(option);
    </script>
</div>

</body>
</html>
```

HelloController

```
package com.henan.hebi.wangzhilong.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@Controller
public class HelloController {
    @GetMapping(value = "/ismobile")
    public String showIsMobile() {
        return "view-IsMobile";
    }

    @GetMapping(value = "/Day")
    public String showDay() {
        return "view-Days";
    }
    @GetMapping(value = "/CreationTime")
    public String showCreationTime() {
        return "view-CreationTime";
    }
```

```
    @GetMapping(value = "/UserLevel")
    public String showUserLevel() {
        return "view-UserLevel";
    }

    @GetMapping(value = "/KwCloud")
    public String showKwCloud() {
        return "view-KwCloud";
    }

    @GetMapping(value = "/CreationHour")
    public String showCreationHour() {
        return "view-CreationHour";
    }

    @GetMapping(value = "/CreationMonth")
    public String showCreationMonth() {
        return "view-CreationMonth";
    }

    @GetMapping(value = "/Comment")
    public String showComment() {
        return "view-Comment";
    }

    @GetMapping(value = "/Score")
    public String showScore() {
        return "view-Score";
    }

}
```

JsonResullt

Model

```
package com.henan.hebi.wangzhilong.model;

import lombok.Data;

@Data
public class JsonResult {
    private String status = null;

    private Object result = null;

    public JsonResult status(String status) {
        this.status = status;
        return this;
    }

}
```

分词代码

```
package fenci
import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
import org.apache.spark.sql.SQLContext
import java.sql.Connection
import java.sql.PreparedStatement
import java.sql.DriverManager
/**
 * 在MySQL中创建数据库，命名为edu8
   view plain  copy
   create  database  edu8  default  CHARACTER  SET  utf8;
   使用edu8库并在MySQL中创建表，命名为kwcloud，包含kw和num字段，字符类型分别为varchar(50)和double。
   view plain  copy
   use  edu8;
   create  table  kwcloud  (kw  varchar(50),  num  double);
 */
object WordCountSpark {
  def main(args: Array[String]) {
    val conf = new SparkConf().setAppName("wordcountspark-test").setMaster("local[2]")
    val sc = new SparkContext(conf)
    val lines = sc.textFile("hdfs://localhost:9000/out2/*")
    val words = lines.flatMap { line => line.split(" ") }
    val pairs = words.map { word => (word, 1) }
    val wordCounts = pairs.reduceByKey(_ + _)
    wordCounts.foreachPartition(Insert)
  }

  def Insert(iterator: Iterator[(String, Int)]): Unit = {
```

```
    var conn: Connection = null
    var ps: PreparedStatement = null
    val database = "myedu"
    val user = "root"
    val password = "000000"
    val conn_str = "jdbc:mysql://localhost:3306/" + database +
"?useSSL=false&characterEncoding=UTF-8&" + "user=" + user + "&password="
+ password
    val sql = "INSERT INTO kwcloud (kw,num) VALUES (?,?)"
    try {
      conn = DriverManager.getConnection(conn_str)
      //print(conn)
      iterator.foreach(data => {
        ps = conn.prepareStatement(sql)
        ps.setString(1,data._1)
        ps.setInt(2,data._2)
        //val conn_str = "jdbc:mysql://localhost:3306/" + database +
"?characterEncoding=UTF-8&" + "user=" + user + "&password=" + password
        ps.executeUpdate()
      })
    } catch {
      case e: Exception => println(e.getMessage())
    } finally {
      if (ps != null) {
        ps.close()
      }
      if (conn != null) {
        conn.close()
      }
    }
  }
```

```
}
```

Dayshive 导入到 dayssql;

1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量

view plain copy

```
create table dayshive(
day int,
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 dayshive 表中。

view plain copy

```
insert into table dayshive
select days, count(1) as num from pinglun group by days order by num desc;
```

查看 Hive 表 dayshive 中的数据

view plain copy

```
select * from dayshive;
```

3.在 Mysql 端口创建 dayssql 表，用于存储 dayshive 中的数据。dayssql 中同样包含两个字段间隔时间及数量

view plain copy

```
create table dayssql(day int,num int);
```

4.使用 Sqoop 命令将 Hive 的 dayshive 表导入到 Mysql 的 dayssql 里。

view plain copy

```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password strongs \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/dayshive/000000_0 \
--input-fields-terminated-by '\t'
```

查看 mysql 中 dayssql 表。

view plain copy

```
select * from dayssql;
```

Creationtimehive 导入到 creationtimesql 2

1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量

view plain copy

```
create table dayshive(
Creationtime varchar(10),
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 Creationtimehive 表中。

view plain copy

```
insert into table Creationtimeshive
select sub(Creationtime,0,4), count(1) as num from pinglun group by Creationtime order by num desc;
```

查看 Hive 表 Creationtimehive 中的数据

view plain copy

```
select * fromCreationtimehive;
```

3.在 Mysql 端口创建 Creationtimesql 表，用于存储 Creationtimehive 中的数据。Creationtimesql 中同样包含两个字段间隔时间及数量

view plain copy

```
create table Creationtimesql(Creationtime varchar,num int);
```

4.使用 Sqoop 命令将 Hive 的 Creationtimehive 表导入到 Mysql 的 Creationtimesql 里。

view plain copy

```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/Creationtimehive/000000_0 \
--input-fields-terminated-by '\t'
```

查看 mysql 中 Creationtimesql 表。

view plain copy

```
select * fromCreationtimesql;
```

Creationhourhive 导入到 creationhoursql 2

1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量

view plain copy

```
create table dayshive(
Creationhour varchar(10),
num int
)
row format delimited
fields terminated by '\t'
```

stored as textfile;

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 Creationhourhive 表中。
view plain copy
insert into table Creationhourshive
select sub(Creationhour,11,3), count(1) as num from pinglun group by Creationhour order by n
um desc;
查看 Hive 表 Creationhourhive 中的数据
view plain copy
select * fromCreationhourhive;
3. 在 Mysql 端口创建 Creationhoursql 表，用于存储 Creationhourhive 中的数据。Creationhoursql 中同样包含两个字段间隔时间及数量
view plain copy
create table Creationhoursql(Creationhour varchar,num int);
4.使用 Sqoop 命令将 Hive 的 Creationhourhive 表导入到 Mysql 的 Creationhoursql 里。
view plain copy
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/Creationhourhive/000000_0 \
--input-fields-terminated-by '\t'
查看 mysql 中 Creationhoursql 表。
view plain copy
select * fromCreationhoursql;

Creationmonthhive 导入到 creationmonthsql 2
1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量
view plain copy
create tablemonthhive(
Creationmonth varchar(10),
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 Creationmonthhive 表中。
view plain copy
insert into table Creationmonthshive
select sub(Creationmonth,11,3), count(1) as num from pinglun group by Creationmonth order
by num desc;

查看 Hive 表 Creationmonthhive 中的数据
view plain copy
select * fromCreationmonthhive;
3.在 Mysql 端口创建 Creationmonthsql 表，用于存储 Creationmonthhive 中的数据。Creationmonthsql 中同样包含两个字段间隔时间及数量
view plain copy
create table Creationmonthsql(Creationmonth varchar,num int);
4.使用 Sqoop 命令将 Hive 的 Creationmonthhive 表导入到 Mysql 的 Creationmonthsql 里。
view plain copy
```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/Creationmonthhive/000000_0 \
--input-fields-terminated-by '\t'
```
查看 mysql 中 Creationmonthsql 表。
view plain copy
select * fromCreationmonthsql;

scorehive 导入到 scoresql 2
1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量
view plain copy
```
create tablemonthhive(
score varchar(10),
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 scorehive 表中。
view plain copy
```
insert into table scoreshive
Select score, count(1) as num from pinglun group by score order by num desc;
```
查看 Hive 表 scorehive 中的数据
view plain copy
select * from scorehive;
3.在 Mysql 端口创建 scoresql 表，用于存储 scorehive 中的数据。scoresql 中同样包含两个字段间隔时间及数量
view plain copy
create table scoresql(score varchar,num int);

4.使用 Sqoop 命令将 Hive 的 scorehive 表导入到 Mysql 的 scoresql 里。
view plain copy
```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/scorehive/000000_0 \
--input-fields-terminated-by '\t'
```
查看 mysql 中 scoresql 表。
view plain copy
```
select * from scoresql;
```

userlevelhive 导入到 userlevelsql 2
1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量
view plain copy
```
create tablemonthhive(
userlevel varchar(10),
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 userlevelhive 表中。
view plain copy
```
insert into table userlevelshive
Select userlevel, count(1) as num from pinglun group by userlevel order by num desc;
```
查看 Hive 表 userlevelhive 中的数据
view plain copy
```
select * from userlevelhive;
```
3.在 Mysql 端口创建 userlevelsql 表，用于存储 userlevelhive 中的数据。userlevelsql 中同样包含两个字段间隔时间及数量
view plain copy
```
create table userlevelsql(userlevel varchar,num int);
```
4.使用 Sqoop 命令将 Hive 的 userlevelhive 表导入到 Mysql 的 userlevelsql 里。
view plain copy
```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
```

```
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/userlevelhive/000000_0 \
--input-fields-terminated-by '\t'
```

查看 mysql 中 userlevelsql 表。

view plain copy

```
select * from userlevelsql;
```

userlevelhive 导入到 userlevelsql 2

1.在 Hive 中，创建用户评论周期分析结果表 dayshive，包含两个字段多久评论及数量

view plain copy

```
create tablemonthhive(
userlevel varchar(10),
num int
)
row format delimited
fields terminated by '\t'
stored as textfile;
```

2.对用户评论间隔时间进行统计，并将统计结果导入到 Hive 中的 userlevelhive 表中。

view plain copy

```
insert into table userlevelshive
Select userlevel, count(1) as num from pinglun group by userlevel order by num desc;
```

查看 Hive 表 userlevelhive 中的数据

view plain copy

```
select * from userlevelhive;
```

3.在 Mysql 端口创建 userlevelsql 表，用于存储 userlevelhive 中的数据。userlevelsql 中同样包含两个字段间隔时间及数量

view plain copy

```
create table userlevelsql(userlevel varchar,num int);
```

4.使用 Sqoop 命令将 Hive 的 userlevelhive 表导入到 Mysql 的 userlevelsql 里。

view plain copy

```
sqoop export \
--connect jdbc:mysql://localhost:3306/edu4out?characterEncoding=UTF-8 \
--username root \
--password 000000 \
--table dayssql \
--export-dir /user/hive/warehouse/edu4.db/userlevelhive/000000_0 \
--input-fields-terminated-by '\t'
```

查看 mysql 中 userlevelsql 表。

view plain copy

```
select * from userlevelsql;
```